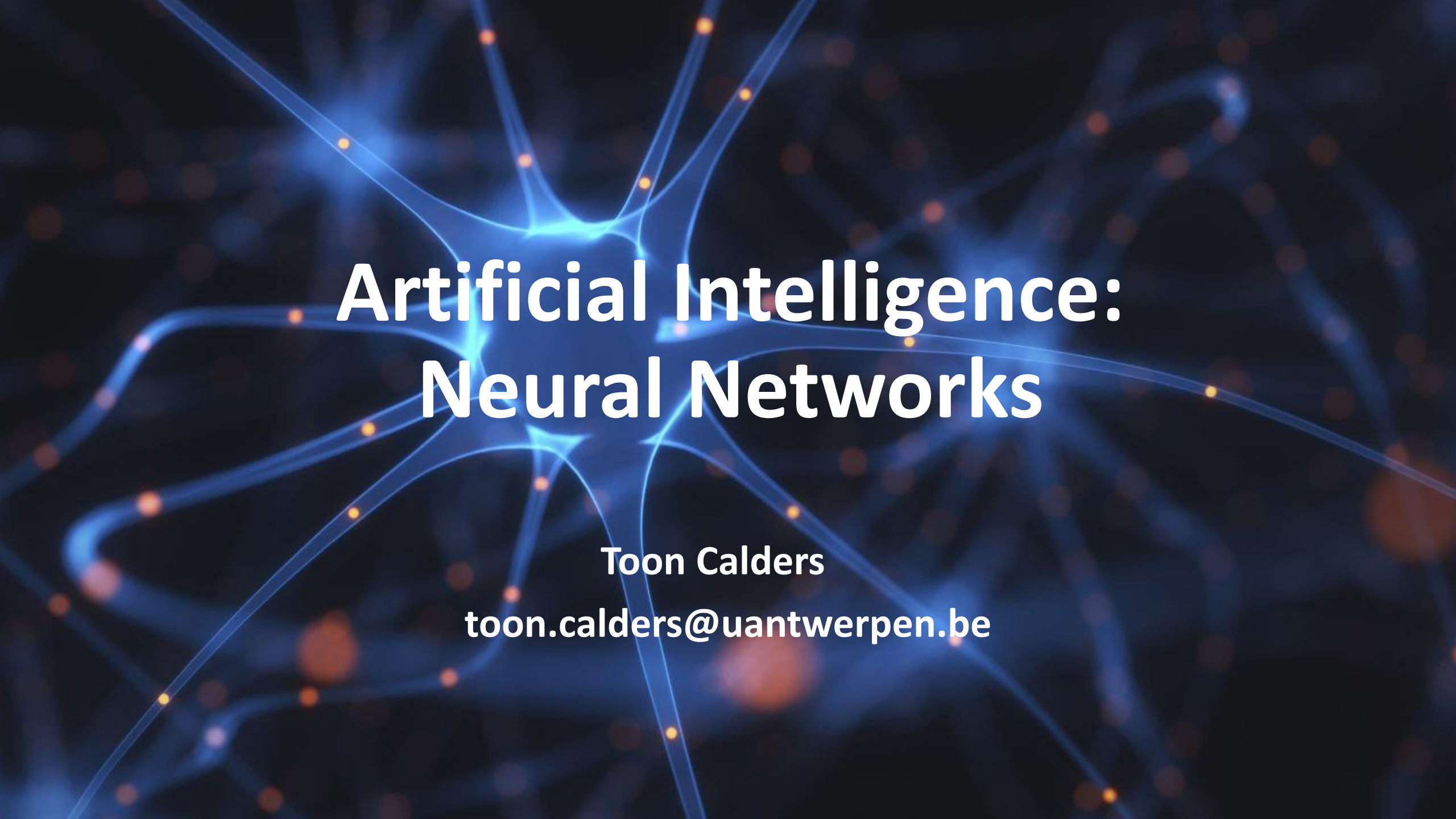


Artificial Intelligence: Neural Networks

Toon Calders

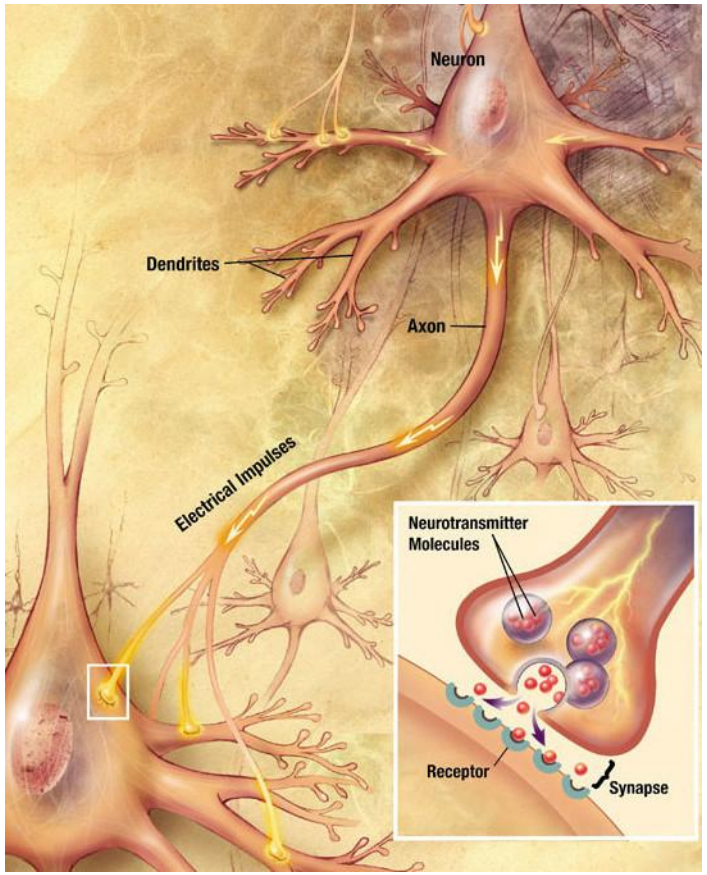
toon.calders@uantwerpen.be

Outline

- Perceptron
- Learning as optimization
- Multi-layer perceptrons
- Deep Neural Networks
- Convolutional Neural Networks

Building Block: Perceptron

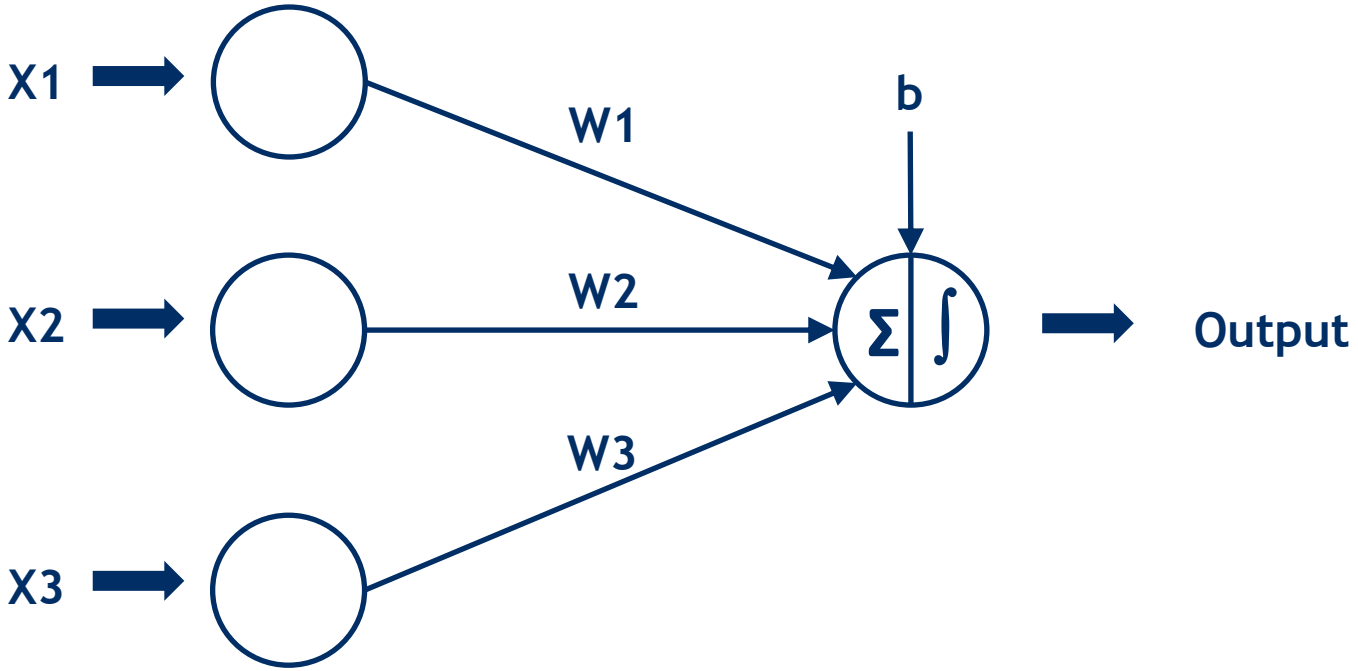
- **A perceptron is a simplified model of a biological neuron.**
  - A neuron receives “input” from different other cells (other neurons or “sensors”)
  - If total input exceeds a threshold, the neuron will “spike” (i.e., emit a signal)
- **Human brain consists out of approximately 86 billion neurons**



Picture from Wikipedia (http://en.wikipedia.org/wiki/Chemical_synapse)

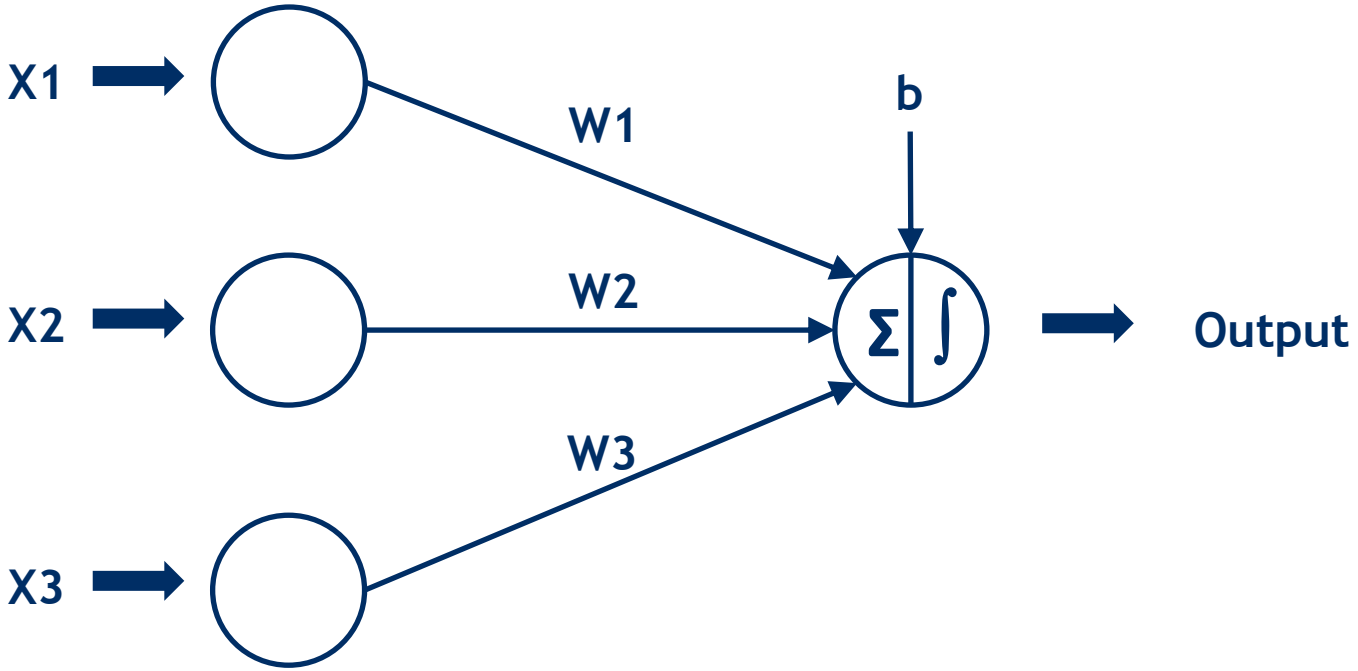
Perceptron

- A perceptron takes inputs and computes a function
- Output depends on *weights* and an *activation function*

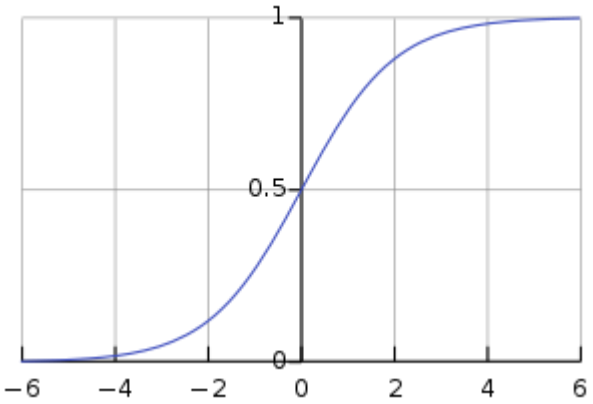


Perceptron

- A perceptron takes inputs and computes a function
- Output depends on *weights* and an *activation function*



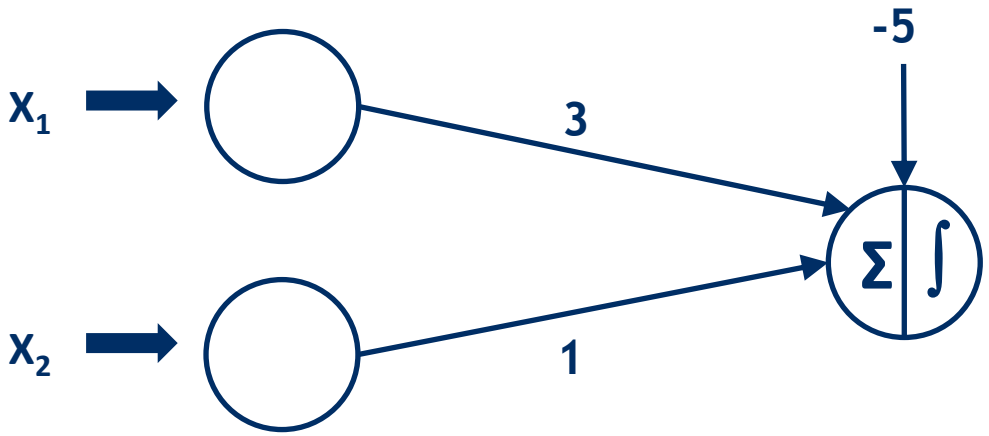
Perceptron

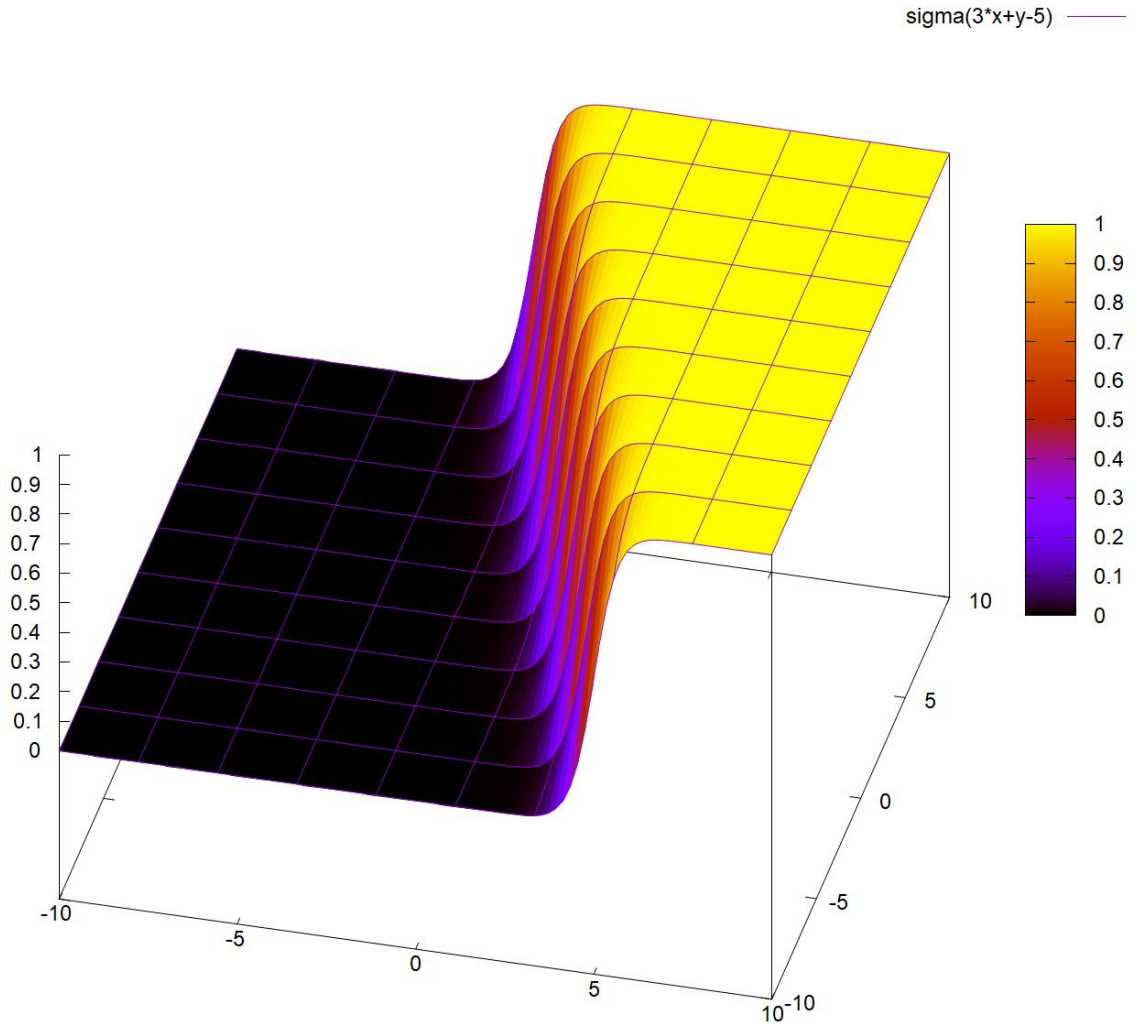


- A perceptron takes inputs and computes a function
- Output depends on *weights* and an *activation function*
- Example of an activation function: $\sigma(x) = \frac{1}{1 + e^{-}x}$
- The perceptron on last page computes the function:

$$\sigma(w_1 x_1 + w_2 x_2 + w_3 x_3 + b)$$

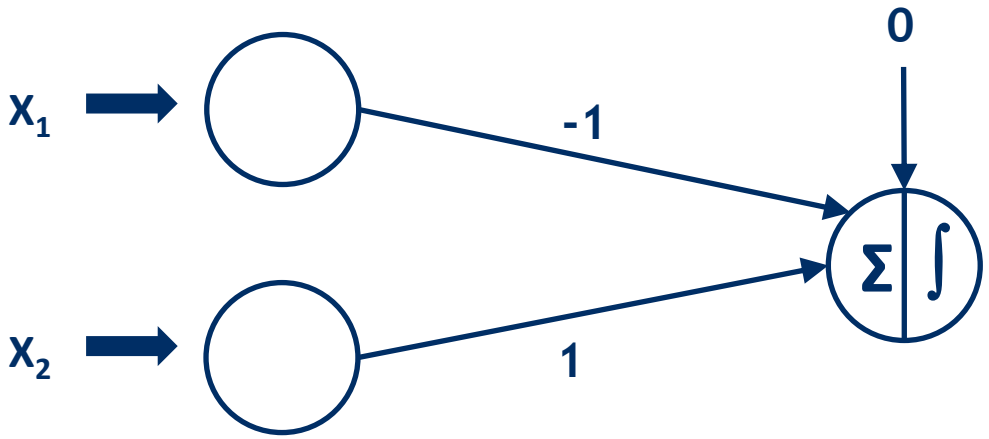
Examples - Perceptrons

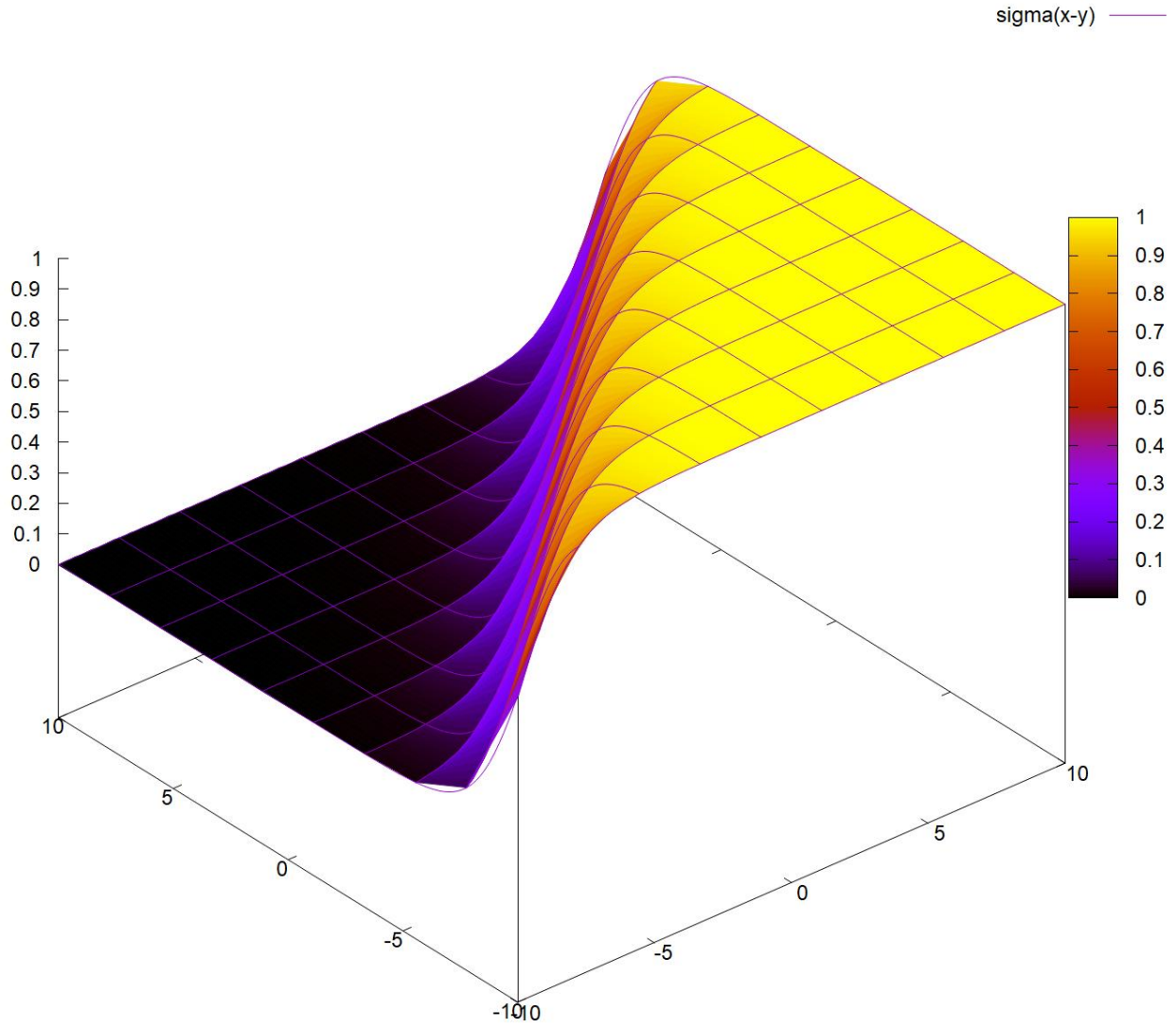




$$\sigma(3x_1 + x_2 - 5)$$

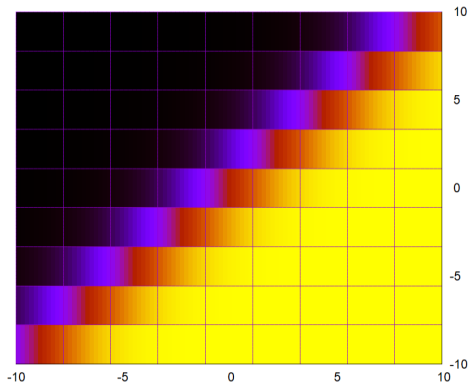
Examples - Perceptrons



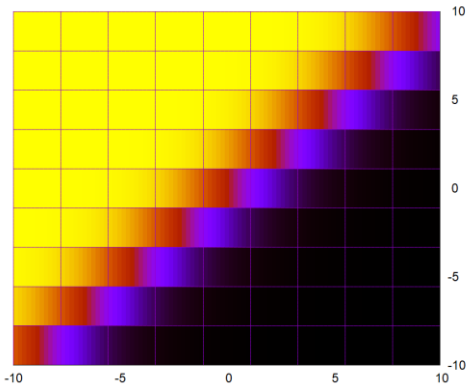


$$\sigma(-x_1 + x_2)$$

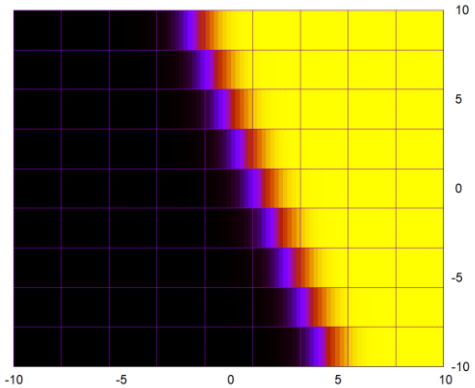
Examples - Perceptrons



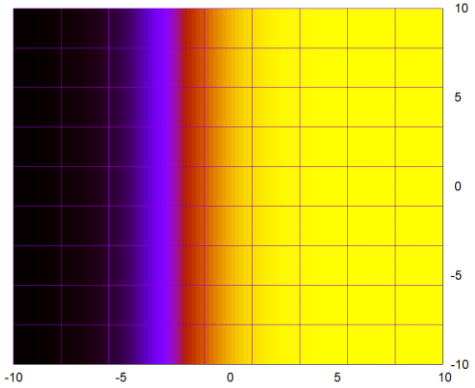
$\sigma(x_1 - x_2)$



$\sigma(-x_1 + x_2)$



$\sigma(3x_1 + x_2 - 5)$



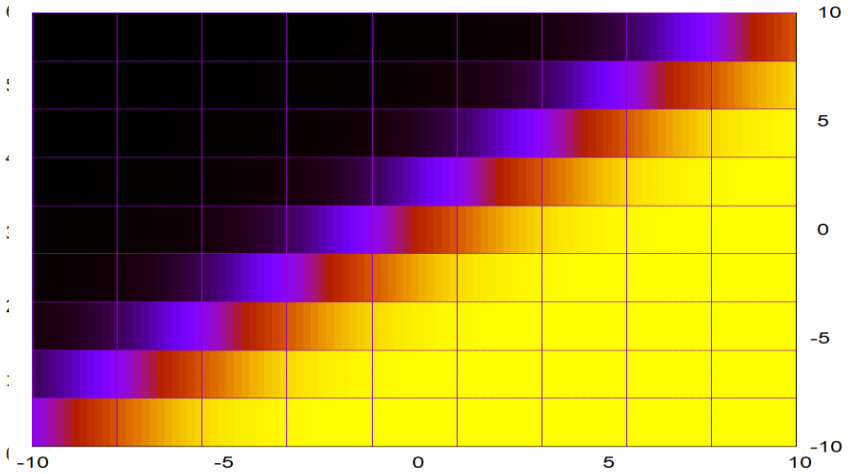
$\sigma(x_1 - 2)$

- **Each choice of w1, w2, b corresponds to another function, mapping (x,y) to a number**

Perceptron Learning - Algorithm

- **Given a dataset, learn the parameters of a perceptron that produces similar output**

x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1



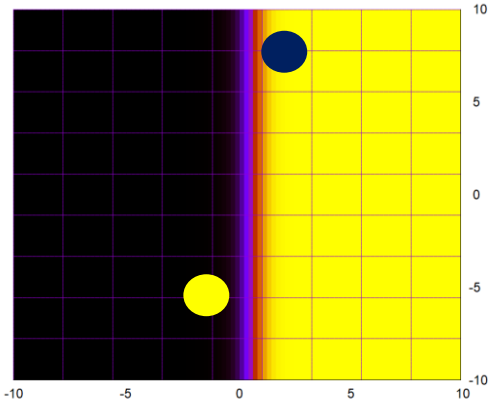
$$\sigma(x_1 - x_2)$$

Perceptron Learning - Algorithm

- **Given a dataset, learn the parameters of a perceptron that produces similar output**
- **Perceptron learning = learning weights = training the network**
  - Iteratively refine the model until it fits the examples
- **Training proceeds as follows:**
  - Start with random weights
  - Repeat until “good enough”:

        For each training example (x,y,label):

            slightly change the weights to improve prediction



example

Outline

- Perceptron
- Learning as optimization
- Multi-layer perceptrons
- Deep Neural Networks
- Convolutional Neural Networks

Classification as Optimization

- We have seen a number of ways to score models
- We could build models to optimize these measures directly

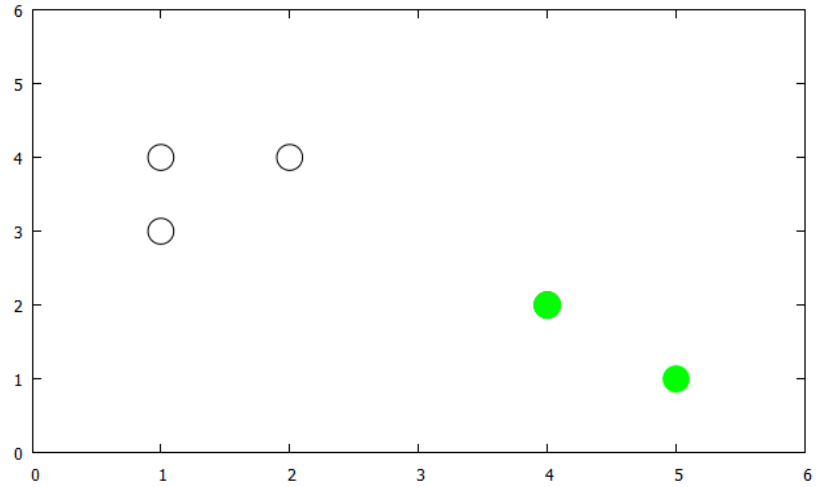
- A *loss-function L* is defined that quantifies how good/bad a prediction is.
- The *Risk* R(h) of a hypothesis h is defined as

  $R(h) := E[L(h(x),y)]$
- The *empirical risk* is defined with respect to a test set $\{(x_1,y_1), \ldots, (x_n,y_n)\}$ and is defined as:
- $R_{emp}(h) := \sum_{i=1}^{n} L(h(x_i), y_i)$

Perceptron Learning - Algorithm

- Given a dataset, learn the parameters of the perceptron

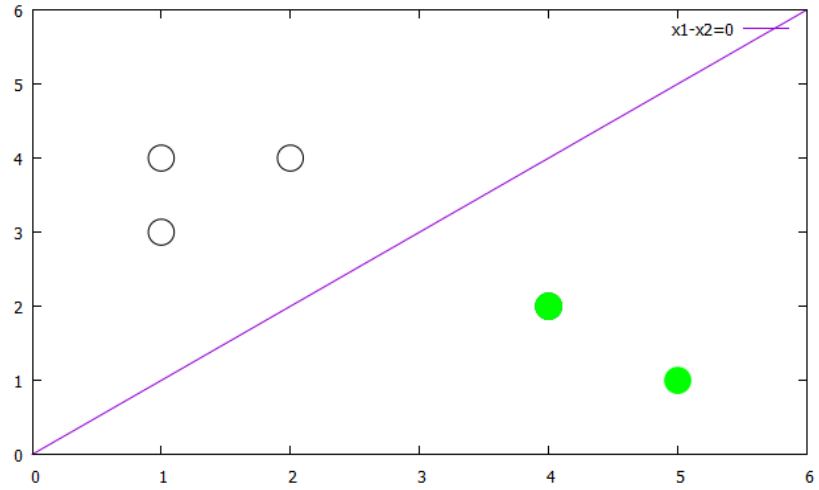
x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1



Perceptron Learning - Algorithm

- **Given a dataset, learn the parameters of the perceptron**

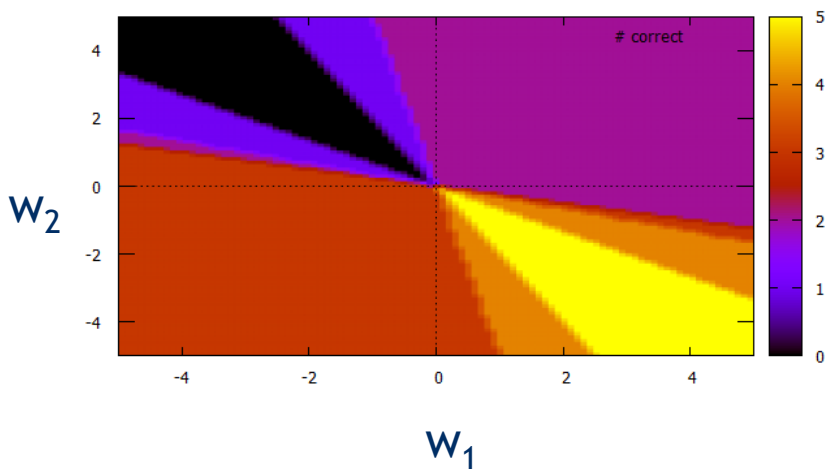
x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

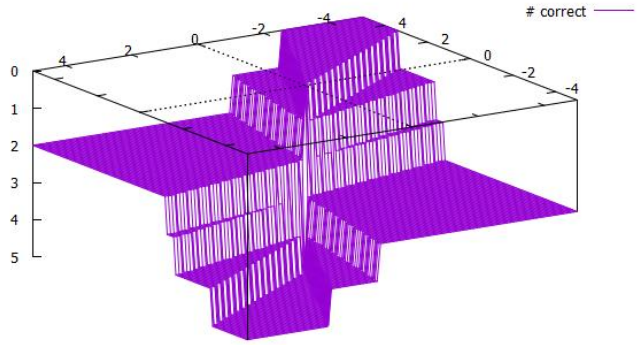


- **Good solution in this case:**
  - w_1=1, w_2=-1, b=0

Perceptron Learning as Optimization

- **Suppose we maximize number of correct predictions**
- **Problem:** (plots show empirical risk in function of w1, w2 when b=0)





- **Surface has everywhere derivative = 0**
- **Hard to optimize! (no gradient descent, e.g.)**

Loss Function – Quadratic Loss

- We sum over the loss of the individual examples:
  - $Loss(D) = \sum_{x,y \in D} L\ (h(\boldsymbol{x}), \boldsymbol{y})$

- In case of multiple output nodes y:

$$Loss(D) = \sum_{\boldsymbol{x},\boldsymbol{y} \in D} \sum_{i} L(h(\boldsymbol{x}), y_i)$$

- Quadratic loss:

$$L(h(\boldsymbol{x}), \boldsymbol{y}) = \frac{1}{2}(h(\boldsymbol{x}) - y)^2$$

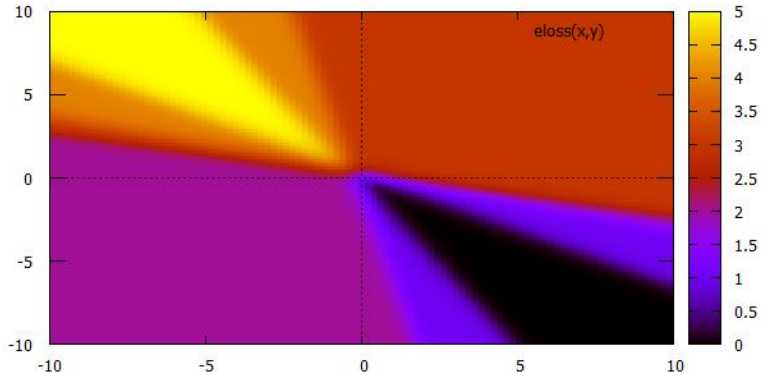
Loss Function – Cross-entropy

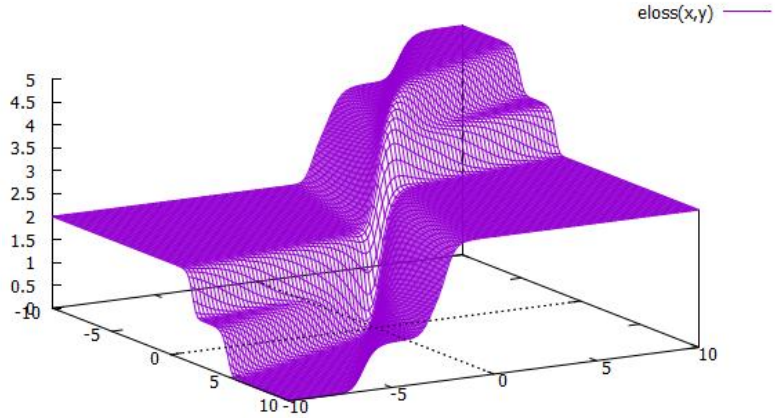
- $L(h(\boldsymbol{x}), y) = -(y \log(h(\boldsymbol{x})) + (1 - y)\log(1 - h(\boldsymbol{x})))$
  - If y=1: -log(h(**x**))
  - If y=0: -log(1-h(**x**))

- **Learning a perceptron with Sigmoid activation function corresponds to *logistic regression*!**

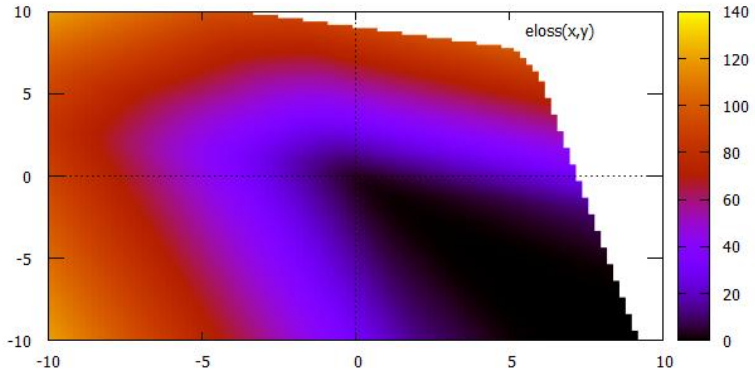
Illustration: Different Loss Functions

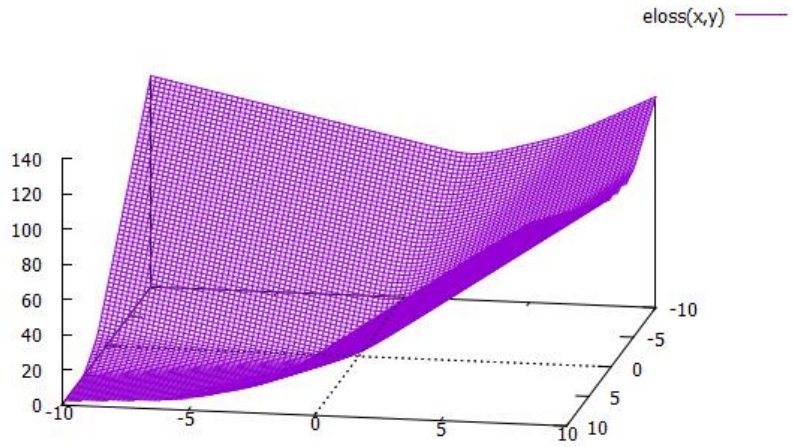
- **Quadratic loss**





- **Logloss**





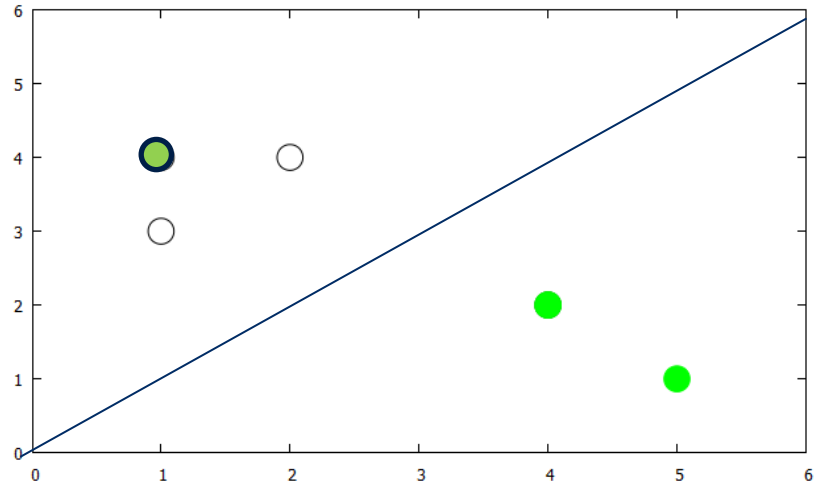
Outline

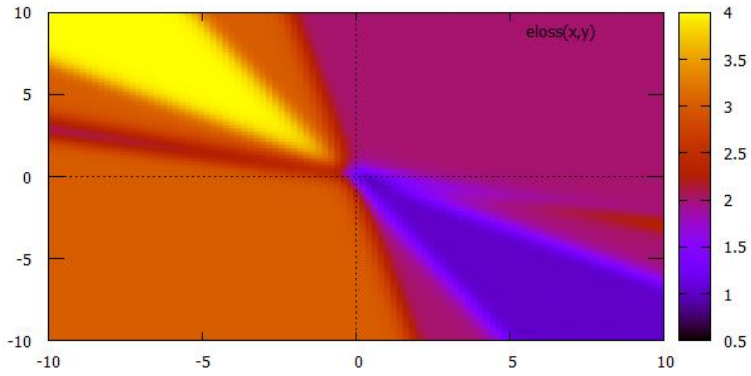
- Perceptron
- Learning as optimization
- Multi-layer perceptrons
- Deep Neural Networks
- Convolutional Neural Networks

Perceptrons are Limited

- **Given a dataset, learn the parameters of the perceptron**

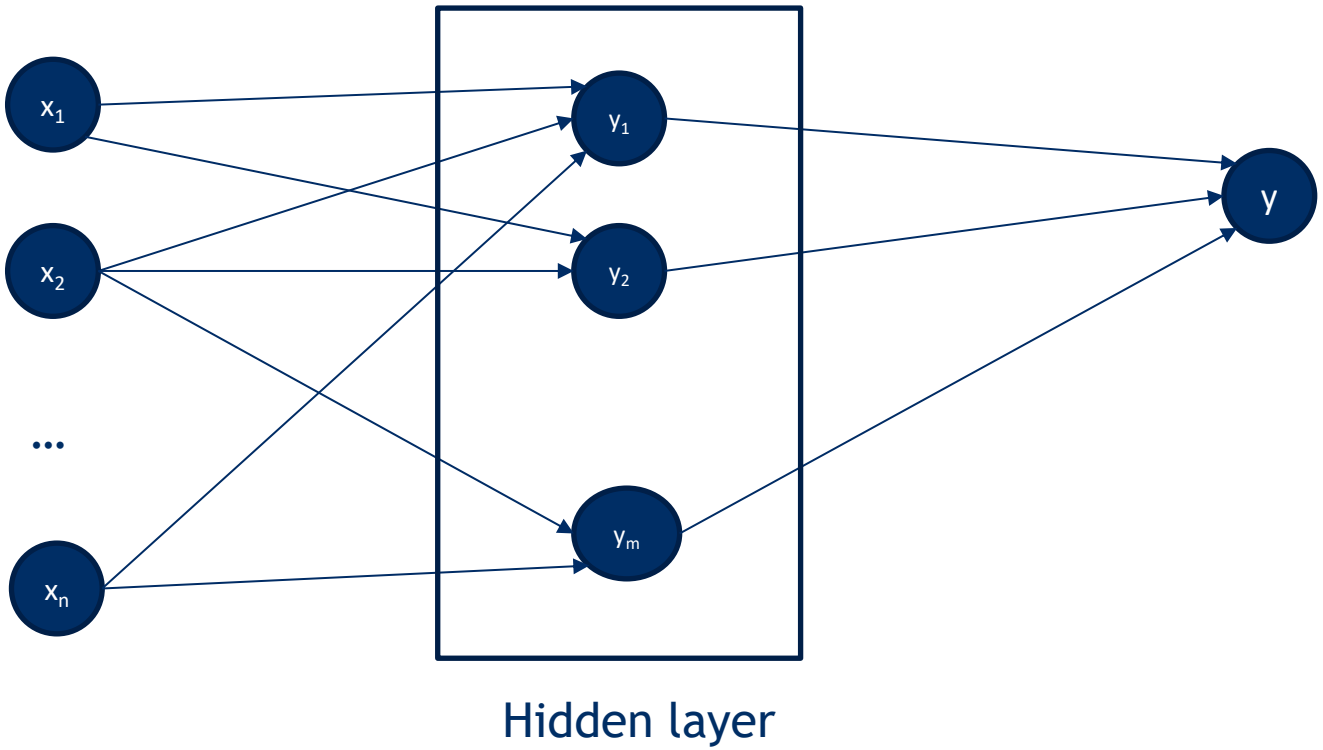
x_1	x_2	y
1	3	0
1	4	1
2	4	0
4	2	1
5	1	1



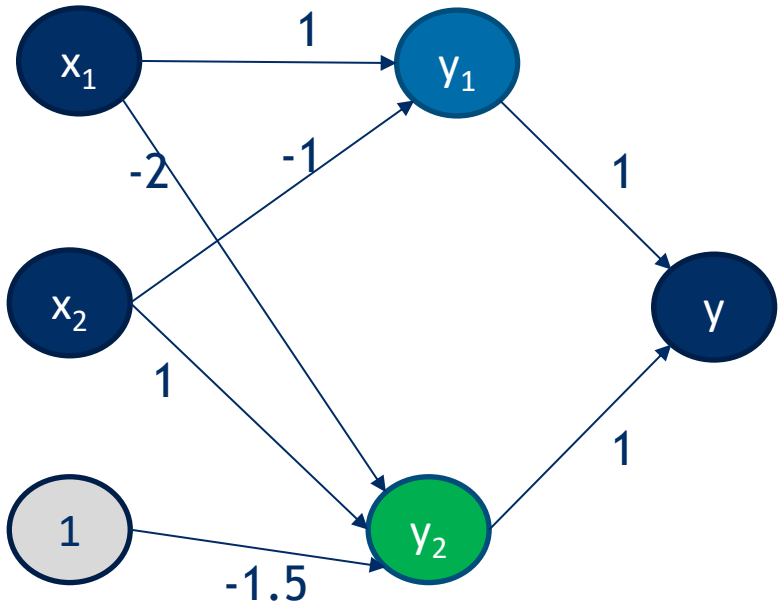


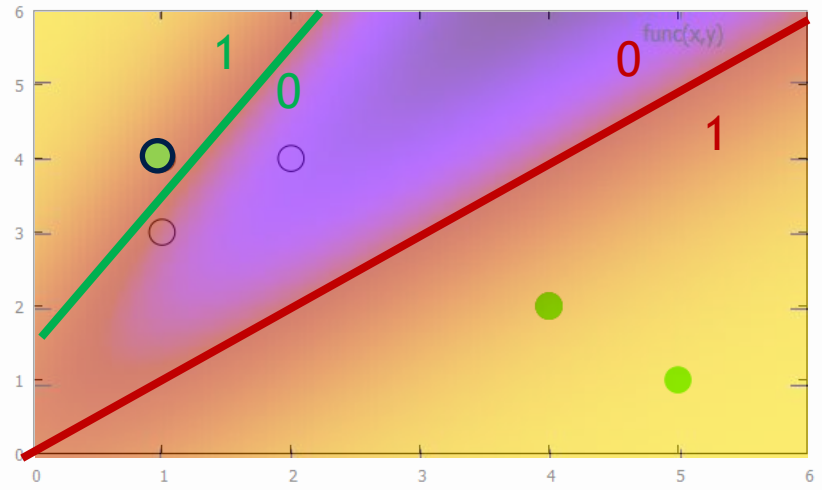
Extending to Multiple Perceptrons

- **We can combine multiple perceptrons to create a more complex *neural network***



Hidden Layer Acts as Space Partitioner





Some Examples of MLPs

- http://playground.tensorflow.org

Optimization Problem

- **Find weights that optimize empirical loss:**

$$R_{emp} = \sum_{\mathbf{x},y \in \mathcal{D}} \mathcal{L}(h_\theta(\mathbf{x}), y)$$

- **To optimzie this function, we will use Gradient descent**
  - Gradient $\nabla_\theta R_{Emp}\left(\theta^{(n)}\right)$ = direction of highest increase of R_{Emp} given current weights $\theta^{(n)}$
  - Move in the opposite direction of the gradient

Gradient ≈ weight update

$$\theta^{(n+1)} = \theta^{(n)} - \lambda \nabla_\theta \left(R_{Emp}\left(\theta^{(n)}\right)\right)$$

New weights

Current weights

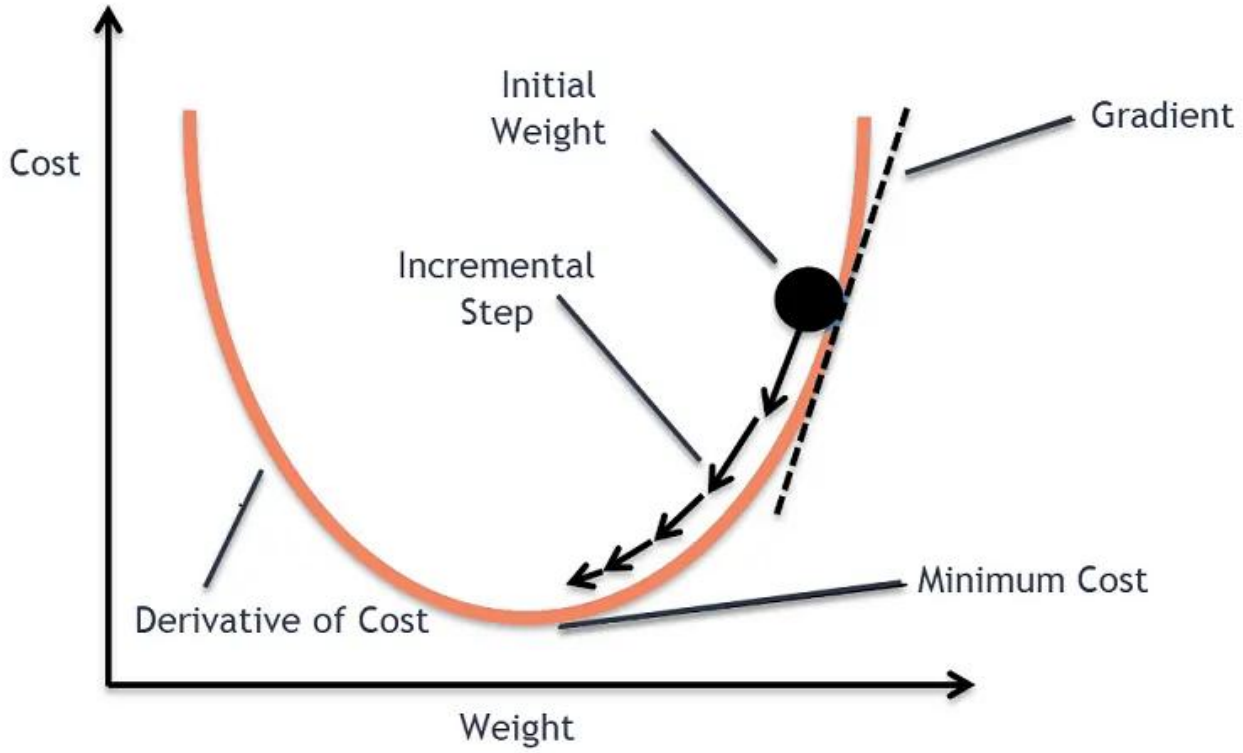
Learning rate

Optimization: Gradient Descent

- **Minimize *R(ϑ)* in function of *ϑ*.**

$$\nabla_\theta(R(\theta)) = \begin{pmatrix} \frac{\partial R(\theta)}{\partial \theta_1} \\ \dots \\ \frac{\partial R(\theta)}{\partial \theta_m} \end{pmatrix}$$

$$\theta^{(n+1)} = \theta^{(n)} - \lambda \nabla_\theta \left(R\left(\theta^{(n)}\right)\right)$$



- **Continue until local minimum**
  - *Locally* optimal solution

Computing the Gradient

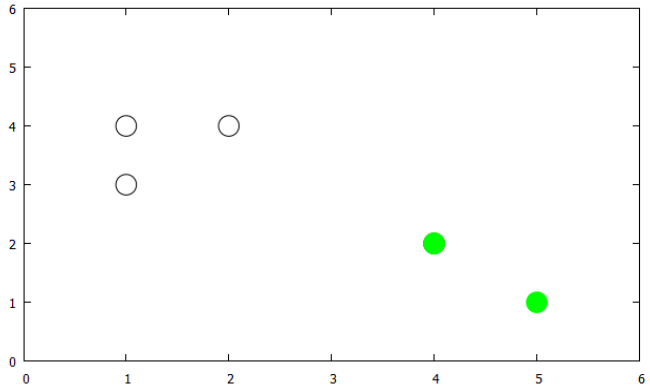
- **So, we need to compute, for each weight w:**

$$\frac{\partial R_{Emp}}{\partial w} = \frac{\partial \sum_{\mathbf{x},y \in \mathcal{D}} \mathcal{L}(h(\mathbf{x}), y)}{\partial w}$$

$$= \sum_{\mathbf{x},y \in \mathcal{D}} \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial w}$$

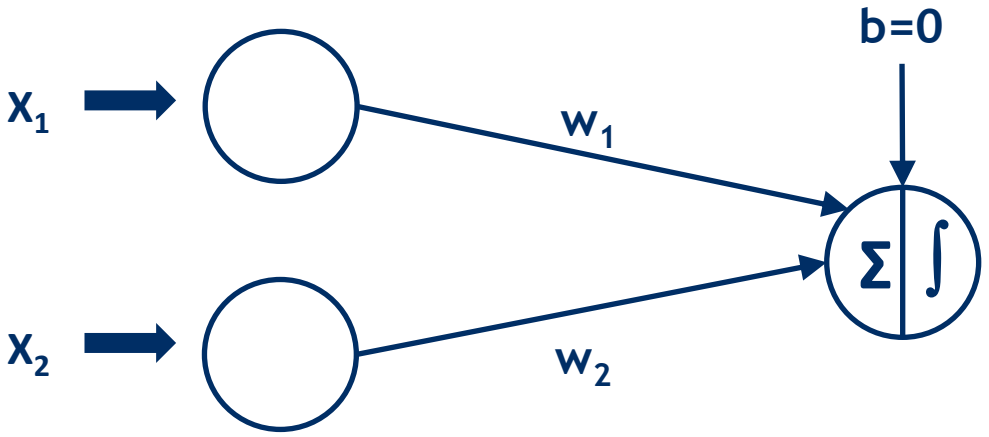
- **Hence, we can compute the derivative example by example and in the end add the gradients**

Example: Perceptron, Quadratic Loss

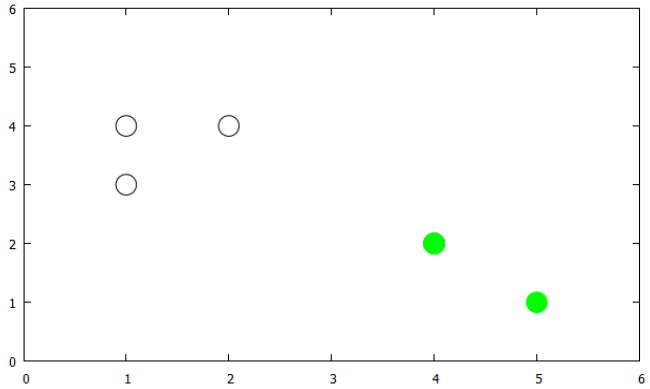


x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

$$h_{(w_1,w_2)}(x_1, x_2) = \sigma(w_1 x_1 + w_2 x_2)$$



Example: Perceptron, Quadratic Loss

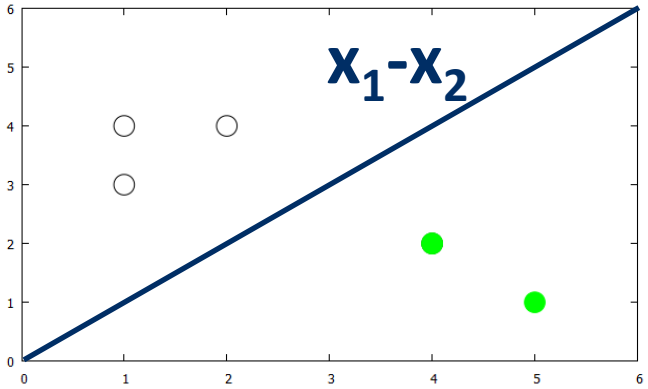


x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

$$h_{(w_1,w_2)}(x_1, x_2) = \sigma(w_1 x_1 + w_2 x_2)$$

$$\begin{aligned} 2 \cdot R_{Emp}(w_1, w_2) \quad = \quad & (\sigma(w_1 + 3w_2))^2 \\ & + (\sigma(w_1 + 4w_2))^2 \\ & + (\sigma(2w_1 + 4w_2))^2 \\ & + (\sigma(4w_1 + 2w_2) - 1)^2 \\ & + (\sigma(5w_1 + w_2) - 1)^2 \end{aligned}$$

Example: Perceptron, Quadratic Loss

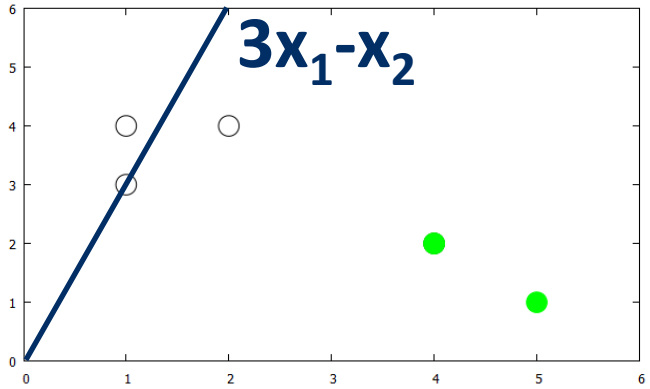


x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

$$h_{(1,-1)}(x_1, x_2) = \sigma(x_1 - x_2)$$

$$\begin{aligned} 2 \cdot R_{Emp}(1,-1) &= (\sigma(1 + 3 \cdot -1))^2 \\ &\quad +(\sigma(1 + 4 \cdot -1))^2 \\ &\quad +(\sigma(2 \cdot 1 + 4 \cdot -1))^2 \\ &\quad +(\sigma(4 \cdot 1 + 2 \cdot -1) - 1)^2 \\ &\quad +(\sigma(5 \cdot 1 - 1) - 1)^2 \\ &= 0.045201 \end{aligned}$$

Example: Perceptron, Quadratic Loss



x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

$$h_{(3,-1)}(x_1, x_2) = \sigma(3x_1 - x_2)$$

$$\begin{aligned} 2 \cdot R_{Emp}(3,-1) &= (\sigma(3 + 3 \cdot -1))^2 \\ &\quad +(\sigma(3 + 4 \cdot -1))^2 \\ &\quad +(\sigma(2 \cdot 3 + 4 \cdot -1))^2 \\ &\quad +(\sigma(4 \cdot 3 + 2 \cdot -1) - 1)^2 \\ &\quad +(\sigma(5 \cdot 3 - 1) - 1)^2 \\ &= 1.098133 \end{aligned}$$

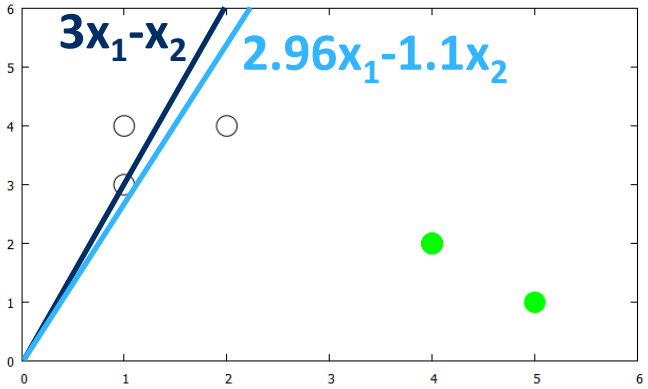
Gradient, Quadratic Loss

$$
\begin{aligned}
&R_{Emp}(w_1, w_2) \\
&= \frac{1}{2}\Big(\quad (\sigma(w_1 + 3w_2))^2 \\
&\qquad + (\sigma(w_1 + 4w_2))^2 \\
&\qquad + (\sigma(2w_1 + 4w_2))^2 \\
&\qquad + (\sigma(4w_1 + 2w_2) - 1)^2 \\
&\qquad + (\sigma(5w_1 + w_2) - 1)^2\Big)
\end{aligned}
$$

$$
\begin{aligned}
&\frac{\partial R_{Emp}}{\partial w_1}(w_1, w_2) \\
&= \quad (\sigma(w_1 + 3w_2)) \cdot \sigma'(w_1 + 3w_2) \\
&\quad + (\sigma(w_1 + 4w_2)) \cdot \sigma'(w_1 + 4w_2) \\
&\quad + (\sigma(2w_1 + 4w_2)) \cdot \sigma(2w_1 + 4w_2) \cdot 2 \\
&\quad + (\sigma(4w_1 + 2w_2) - 1) \cdot \sigma'(4w_1 + 2w_2) \cdot 4 \\
&\quad + (\sigma(5w_1 + w_2) - 1) \cdot \sigma'(5w_1 + w_2) \cdot 5
\end{aligned}
$$

$$
\begin{aligned}
&\frac{\partial R_{Emp}}{\partial w_2}(w_1, w_2) \\
&= \quad (\sigma(w_1 + 3w_2)) \cdot \sigma'(w_1 + 3w_2) \cdot 3 \\
&\quad + (\sigma(w_1 + 4w_2)) \cdot \sigma'(w_1 + 4w_2) \cdot 4 \\
&\quad + (\sigma(2w_1 + 4w_2)) \cdot \sigma(2w_1 + 4w_2) \cdot 4 \\
&\quad + (\sigma(4w_1 + 2w_2) - 1) \cdot \sigma'(4w_1 + 2w_2) \cdot 2 \\
&\quad + (\sigma(5w_1 + w_2) - 1) \cdot \sigma'(5w_1 + w_2)
\end{aligned}
$$

Example: Perceptron, Quadratic Loss



x_1	x_2	y
1	3	0
1	4	0
2	4	0
4	2	1
5	1	1

$$h_{(3,-1)}(x_1, x_2) = \sigma(3x_1 - x_2)$$

$$R_{Emp}(3, -1) \approx 0.549$$

$$\nabla R_{Emp}(3, -1) \approx (0.363, 0.957)$$

- **Updated weights: (learning rate = 0.1)**

  w_1 = 3-0.0363 ≈ 2.96

  w_2 = -1-0.0957 ≈ -1.1

$$R_{Emp}(2.96, -1.1) \approx 0.447$$

Computing the Gradient: General Case

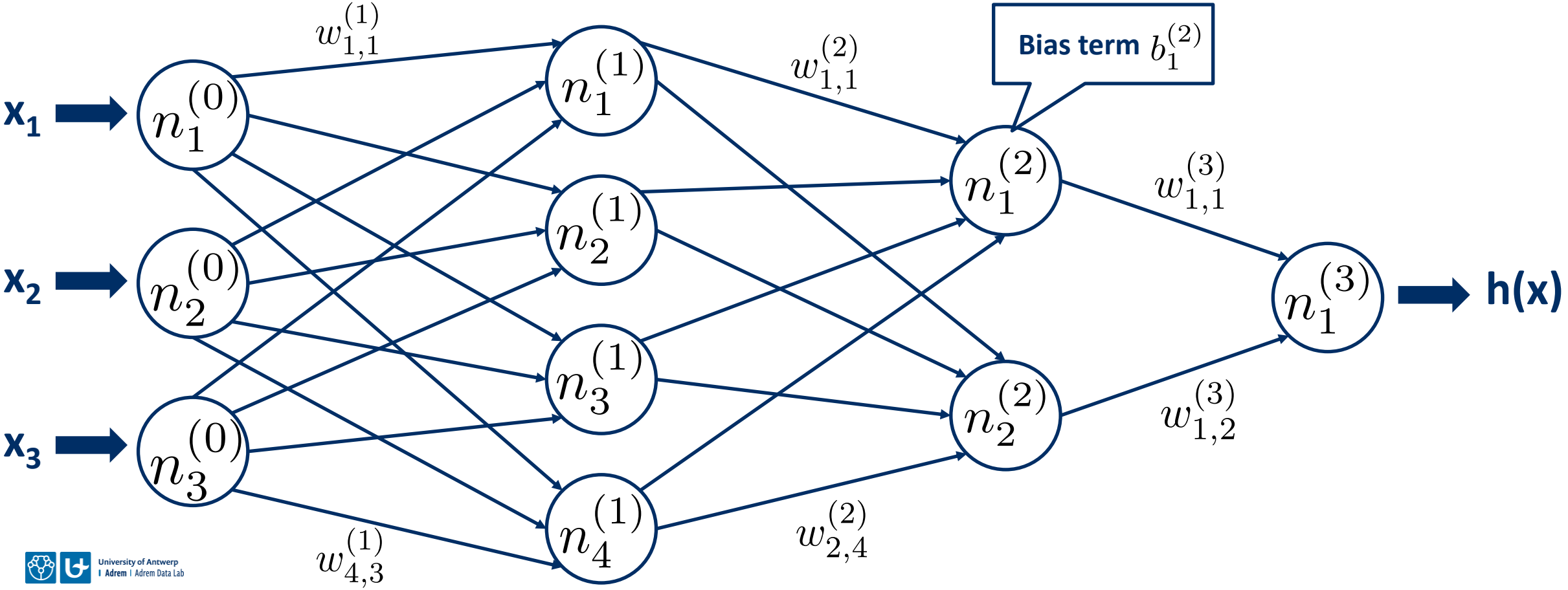
- **Notation:**
  - Node j at layer i: $n_j^{(i)}$
  - Value of the node: $a_j^{(i)}$
  - Weight of the connection: $a_k^{(i-1)} \rightarrow a_j^{(i)} \;:\; w_{jk}^{(i)}$
  - Bias term of node: $b_j^{(i)}$

  - Input layer 0: $a_i^{(0)} = x_i$
  - Output layer: $h_\theta(\mathbf{x}) = \mathbf{a}^{(\ell)}$

θ = vector of all weights and biases

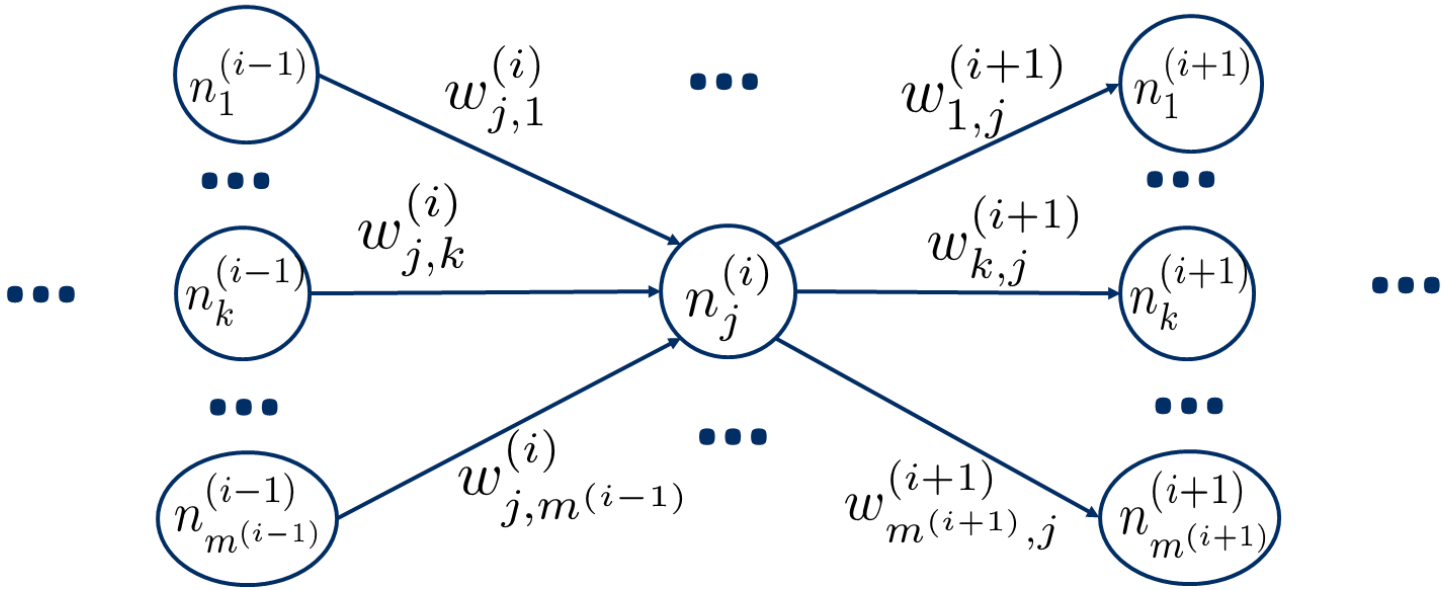
Computing the Gradient: General Case

- **Network with 3 inputs, 2 hidden layers with 4 and 2 nodes, and one output layer with one node:**



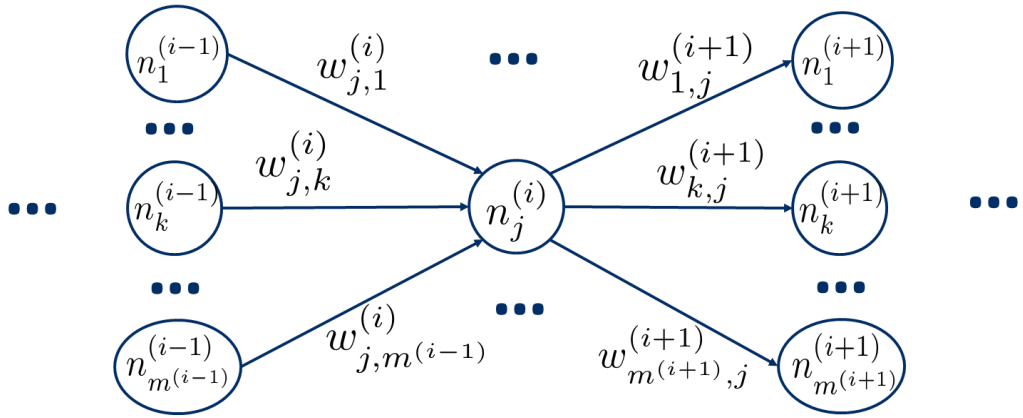
Notation

- **Pre-activation:** $z_j^{(i)} = \sum_k w_{j,k}^{(i)} \cdot a_k^{(i-1)} + b_j^{(i)}$
- **Activation:** $a_j^{(i)} = \sigma\left(z_j^{(i)}\right)$



Gradient Descent – General Case

- We need to compute gradient with respect to all weights w
- Use chain rule



General chain rule:

$$\frac{\partial}{\partial x_i} f(g_1(\mathbf{x}), \ldots, g_k(\mathbf{x})) = \sum_{j=1}^{k} \frac{\partial f}{\partial u_j}(g_1(\mathbf{x}), \ldots, g_k(\mathbf{x})) \frac{\partial g_j(\mathbf{x})}{\partial x_i}$$

Derivative of *f* w.r.t. its *j*th input

Gradient Descent – General Case

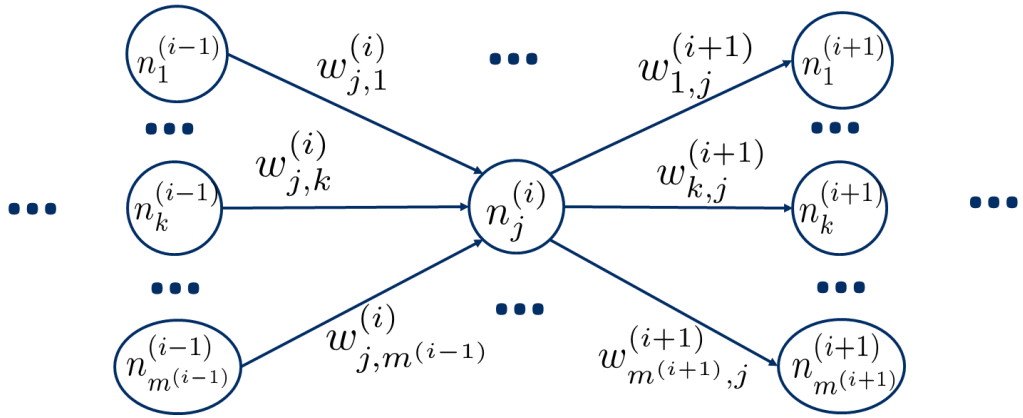
- **We need to compute gradient with respect to all weights w**
- **Trick: use chain rule:**

$$\frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial w_{j,k}^{(i)}} = \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_j^{(i)}} \frac{\partial z_j^{(i)}}{\partial w_{j,k}^{(i)}}$$

$$= \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_j^{(i)}} a_k^{(i-1)}$$

$$= \delta_j^{(i)} a_k^{(i-1)}$$

We will see later how to compute the delta's



$\mathcal{L}(h(\mathbf{x}), y)$ **is a function of** $z_1^{(i)}, \ldots, z_{m^{(i)}}^{(i)}$

Hence, $$\frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial w_{j,k}^{(i)}} = \sum_l \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_l^{(i)}} \frac{\partial z_l^{(i)}}{\partial w_{j,k}^{(i)}}$$

But for all l≠j, $$\frac{\partial z_l^{(i)}}{\partial w_{j,k}^{(i)}} = 0$$

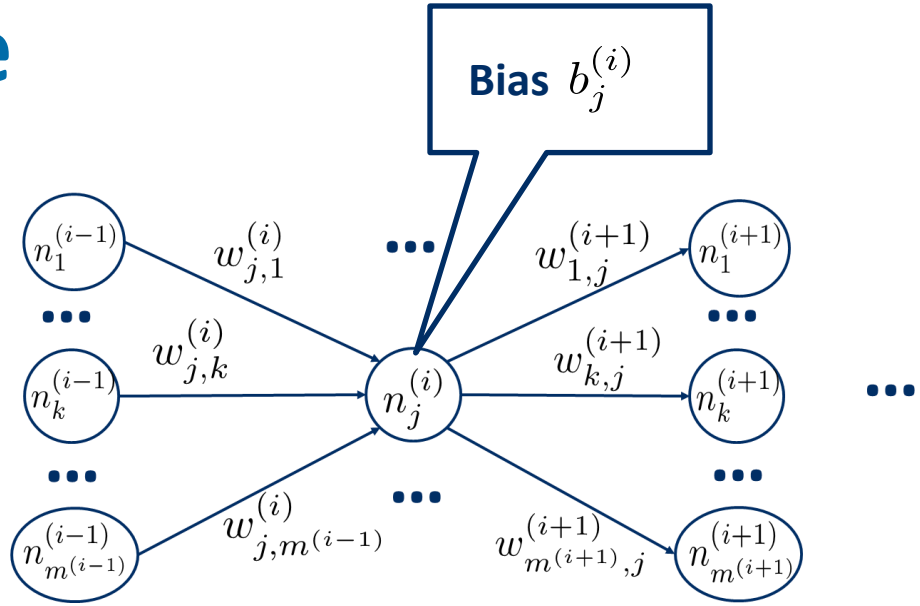
Gradient Descent – General Case

- **We need to compute gradient with respect to all weights w**
- **Use chain rule:**

$$\frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial b_j^{(i)}} = \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_j^{(i)}} \frac{\partial z_j^{(i)}}{\partial b_j^{(i)}}$$

$$= \boxed{\frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_j^{(i)}}} \cdot 1$$

$$= \boxed{\delta_j^{(i)}}$$



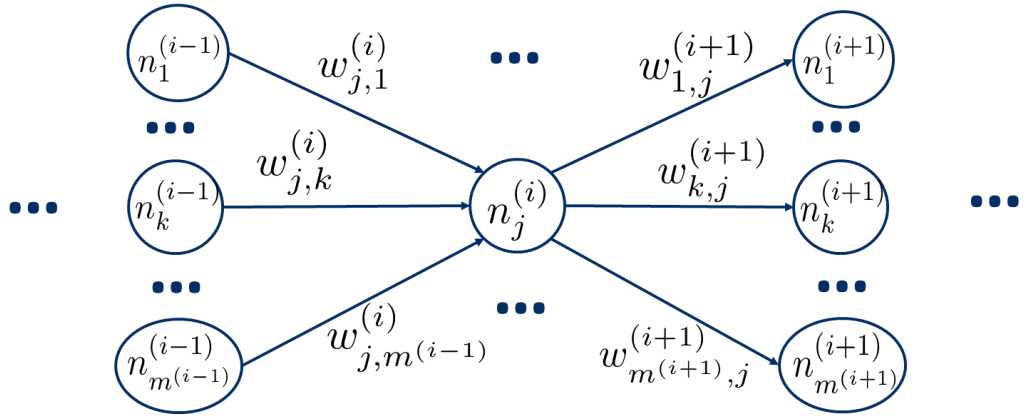
Computing the delta's

- **Last layer:**

$$\begin{aligned}
\delta_i^{(\ell)} &= \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_i^{\ell}} \\
&= \frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}} \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}} \\
&= \frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}} \sigma'\left(z_i^{\ell}\right)
\end{aligned}$$

Computing the delta's

- **Any other layer:**



$$\delta_j^{(i)} = \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_j^{(i)}} = \sum_{k=1}^{m^{(i+1)}} \frac{\partial \mathcal{L}(h(\mathbf{x}), y)}{\partial z_k^{(i+1)}} \frac{\partial z_k^{(i+1)}}{\partial z_j^{(i)}}$$

$\mathcal{L}(h(\mathbf{x}), y)$ **is a function of** $z_1^{(i+1)}, \ldots, z_{m^{(i+1)}}^{(i+1)}$

$$= \sum_{k=1}^{m^{(i+1)}} \delta_k^{(i+1)} w_{k,j}^{(i+1)} \frac{\partial a_j^{(i)}}{\partial z_j^{(i)}} = \sum_{k=1}^{m^{(i+1)}} \delta_k^{(i+1)} w_{k,j}^{(i+1)} \sigma'\left(z_j^{(i)}\right)$$

- **Hence, errors can be *propagated back* from parents to children**

Back-Propagation in a Nutshell

Gradient = sum of gradients for each training example (**x**,y)

$\nabla J(\boldsymbol{x}, y) = \left(\frac{\partial L_y}{\partial w_1}, \ldots, \frac{\partial L_y}{\partial w_k}\right)$ evaluated in point **x**.

Update after batch of examples B: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \lambda \frac{1}{|B|} \sum_{\boldsymbol{x},y \in B} \nabla J(\boldsymbol{x}, y)$

How to compute $\nabla J(\boldsymbol{x}, y)$?

- Forward step:
  - Let $\boldsymbol{x}^n$ be the vector of inputs to a node n
  - For each node n compute z(n) = $\boldsymbol{w}^T \boldsymbol{x}^n$ and a(n) = $\sigma\big(z(n)\big)$
- Backward step:
  - For an output node O, $\delta(O) = L'(a(O), y)\, \sigma'\big(z(O)\big)$
  - For each node N, compute $\delta(N) = \left(\sum_{N' parent\ of\ N} w_{N \to N'}\, \delta(N')\right) \sigma'\big(z(N)\big)$
- If w is the weight on the edge from node A to node B, then $\frac{\partial L}{\partial w}$ equals a(A)δ(B)

Outline

- Perceptron
- Learning as optimization
- Multi-layer perceptrons
- Deep Neural Networks
- Convolutional Neural Networks

Deep Neural Nets - Hyperparameters

- Different choices to make:
  - Network structure
  - Learning rate
  - Activation functions
  - Loss function to optimize

- We can use grid search to optimize (some of) these parameters

Different *Activation Functions*

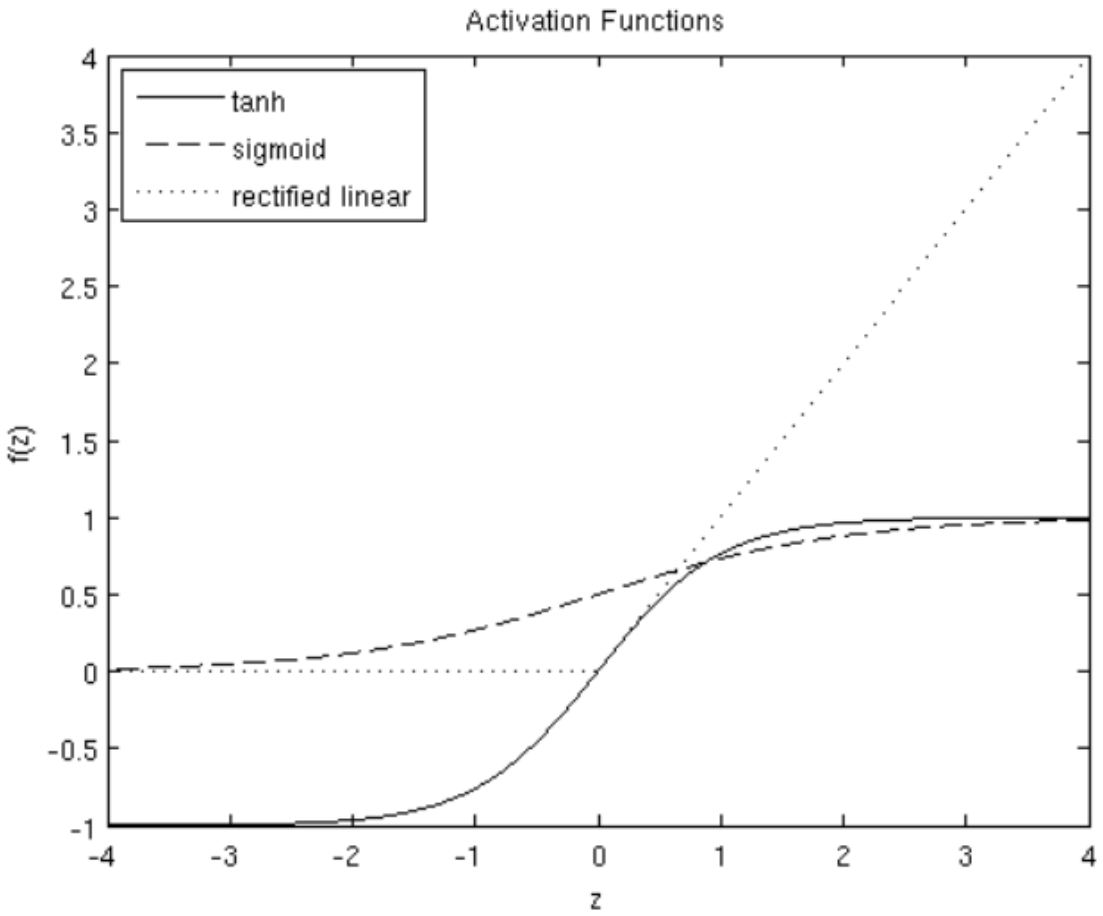
- **tanh:** $\dfrac{e^x - e^{-x}}{e^x + e^{-x}}$

- **Sigmoid:** $\dfrac{1}{1+e^{-x}} = \dfrac{e^x}{e^x+1}$

- **Rectified linear:**

  0 if x<0

  x if x≥0



Which Activation Function to Pick?

- **Sigmoid: produces a score between 0-1**
  - Good for classification problem
  - Disadvantage: gradient near 0 near the ends of the curve

- **ReLU: unbounded for positive values**
  - Advantage: gradient easy to compute and less vulnerable for plateaus
  - Often used in internal layers
  - If output is Gaussian → activation = mean

What if we predict multiple classes?

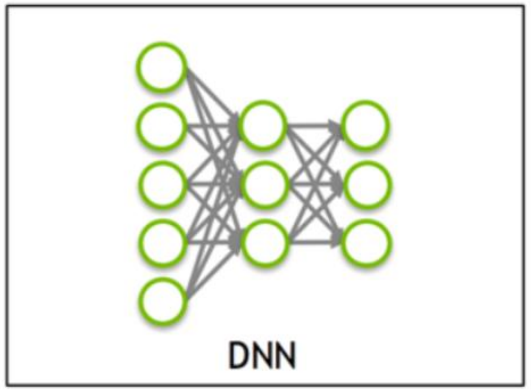
- A so-called soft-max layer is used
- Let $z_i = w^T x$
- $\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$

- That is: Softmax normalizes all z_i such that:
- $0 \leq \text{Softmax}(z_i) \leq 1$
- $\Sigma_i \text{Softmax}(z_i) = 1$

Deep Learning

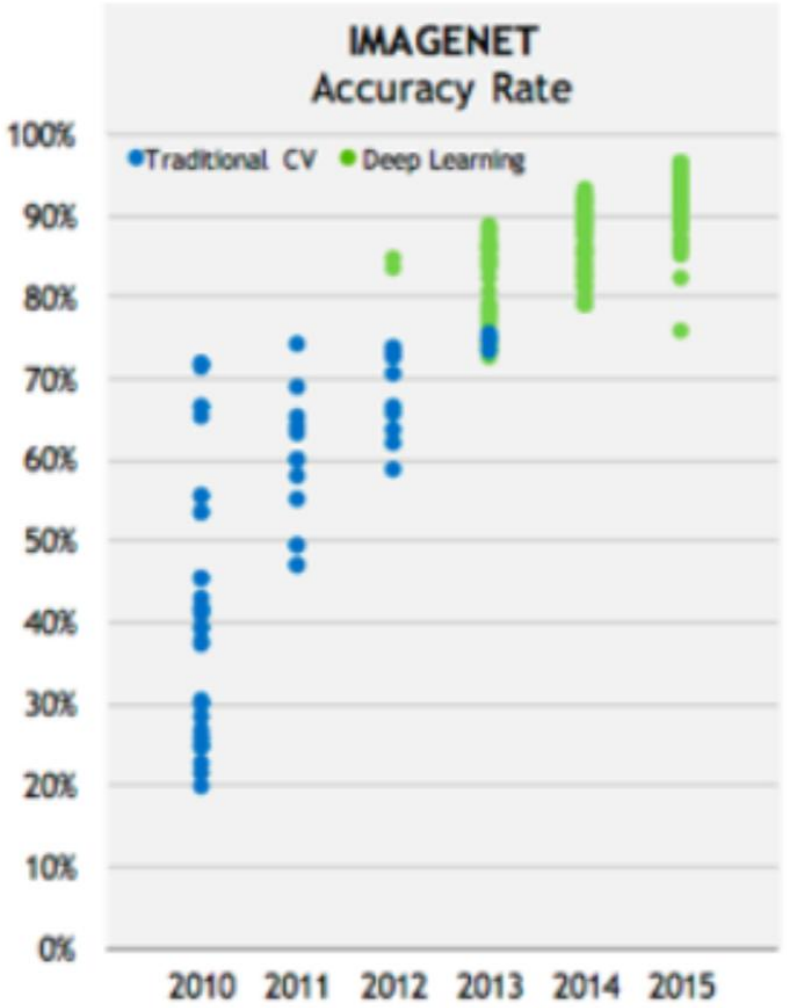
- Deep learning = learning of large NNs
- NNs were already studied in the 60s

- What's new?









What is so great about Neural Networks ?

- **Little to no model bias**
  - They can present any function (if sufficiently large)
  - If there exists a mapping from features to labels, a neural network can in principle learn it
- **Built-in feature engineering**
  - No need for inventing complex features to capture meaningful patterns in the data

- **But, comes at a cost!**
  - Lot and lots of data required!
  - Huge computational demands
  - Expensive hardware

Neural Networks Properties

- **Theorem (Universal Function Approximators). A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.**

- **Practical considerations**
  - Can be seen as learning the features
  - Large number of neurons
    - Danger for overfitting
    - (hence early stopping!)

Universal Function Approximation Theorem*

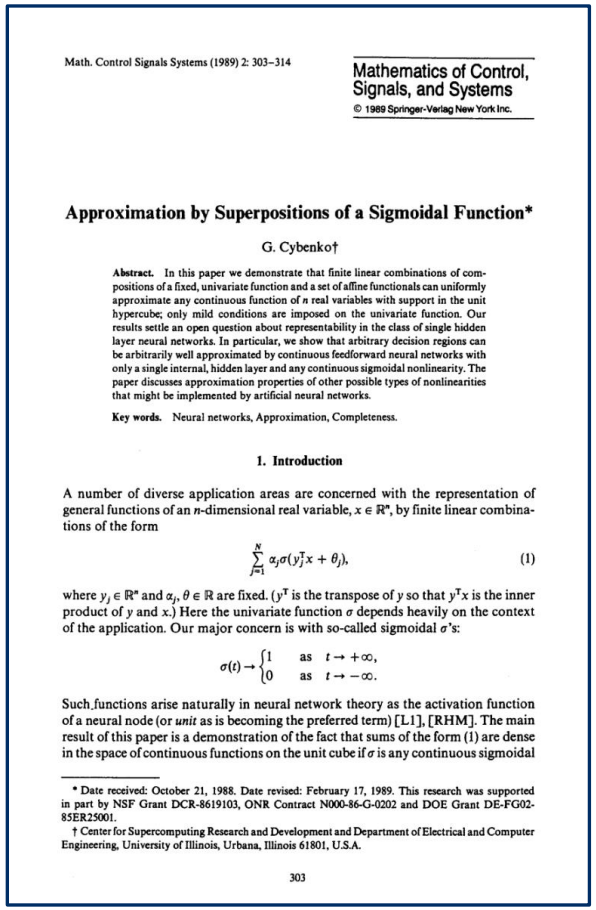
- **<u>In words:</u> Given any continuous function f(x), if a 2-layer neural network has enough hidden units, then there is a choice of weights that allow it to closely approximate f(x).**

<u>Hornik theorem 1:</u> Whenever the activation function is *bounded and nonconstant*, then, for any finite measure μ, standard multilayer feedforward networks can approximate any function in $L^p(\mu)$ (the space of all functions on R^k such that $\int_{R^k} |f(x)|^p d\mu(x) < \infty$) arbitrarily well, provided that sufficiently many hidden units are available.

<u>Hornik theorem 2:</u> Whenever the activation function is *continuous, bounded and nonconstant*, then, for arbitrary compact subsets $X \subseteq R^k$, standard multilayer feedforward networks can approximate any continuous function on X arbitrarily well with respect to uniform distance, provided that sufficiently many hidden units are available.

Cybenko (1989) “Approximations by superpositions of sigmoidal functions”
Hornik (1991) “Approximation Capabilities of Multilayer Feedforward Networks”
Leshno and Schocken (1991) ”Multilayer Feedforward Networks with Non-Polynomial Activation Functions Can Approximate Any Function”

Universal Function Approximation Theorem*



Math. Control Signals Systems (1989) 2: 303–314

Mathematics of Control, Signals, and Systems
© 1989 Springer-Verlag New York Inc.

Approximation by Superpositions of a Sigmoidal Function*

G. Cybenko†

Abstract. In this paper we demonstrate that finite linear combinations of compositions of a fixed, univariate function and a set of affine functionals can uniformly approximate any continuous function of n real variables with support in the unit hypercube; only mild conditions are imposed on the univariate function. Our results settle an open question about representability in the class of single hidden layer neural networks. In particular, we show that arbitrary decision regions can be arbitrarily well approximated by continuous feedforward neural networks with only a single internal, hidden layer and any continuous sigmoidal nonlinearity. The paper discusses approximation properties of other possible types of nonlinearities that might be implemented by artificial neural networks.

Key words. Neural networks, Approximation, Completeness.

1. Introduction

A number of diverse application areas are concerned with the representation of general functions of an n-dimensional real variable, $x \in \mathbb{R}^n$, by finite linear combinations of the form

$$\sum_{j=1}^{N} \alpha_j \sigma(y_j^T x + \theta_j), \tag{1}$$

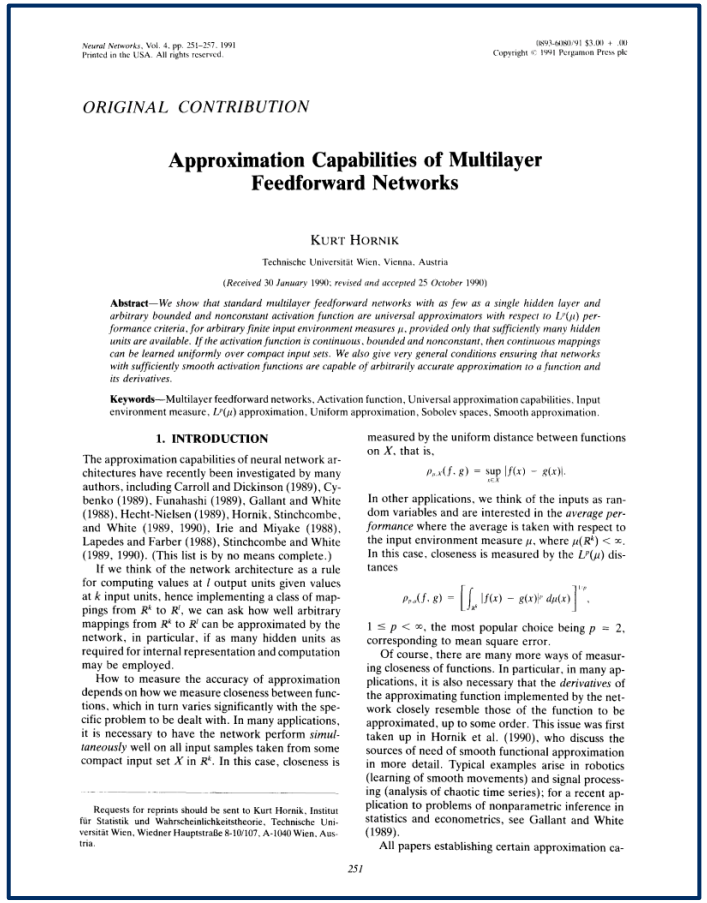
where $y_j \in \mathbb{R}^n$ and $\alpha_j, \theta \in \mathbb{R}$ are fixed. (y^T is the transpose of y so that $y^T x$ is the inner product of y and x.) Here the univariate function σ depends heavily on the context of the application. Our major concern is with so-called sigmoidal σ's:

$$\sigma(t) \to \begin{cases} 1 & \text{as} \quad t \to +\infty, \\ 0 & \text{as} \quad t \to -\infty. \end{cases}$$

Such functions arise naturally in neural network theory as the activation function of a neural node (or *unit* as is becoming the preferred term) [L1], [RHM]. The main result of this paper is a demonstration of the fact that sums of the form (1) are dense in the space of continuous functions on the unit cube if σ is any continuous sigmoidal

* Date received: October 21, 1988. Date revised: February 17, 1989. This research was supported in part by NSF Grant DCR-8619103, ONR Contract N000-86-G-0202 and DOE Grant DE-FG02-85ER25001.

† Center for Supercomputing Research and Development and Department of Electrical and Computer Engineering, University of Illinois, Urbana, Illinois 61801, U.S.A.



Neural Networks, Vol. 4, pp. 251–257. 1991
Printed in the USA. All rights reserved.

0893-6080/91 $3.00 + .00
Copyright © 1991 Pergamon Press plc

ORIGINAL CONTRIBUTION

Approximation Capabilities of Multilayer Feedforward Networks

KURT HORNIK

Technische Universität Wien, Vienna, Austria

(*Received* 30 *January* 1990; *revised and accepted* 25 *October* 1990)

Abstract—*We show that standard multilayer feedforward networks with as few as a single hidden layer and arbitrary bounded and nonconstant activation function are universal approximators with respect to $L^p(\mu)$ performance criteria, for arbitrary finite input environment measures μ, provided only that sufficiently many hidden units are available. If the activation function is continuous, bounded and nonconstant, then continuous mappings can be learned uniformly over compact input sets. We also give very general conditions ensuring that networks with sufficiently smooth activation functions are capable of arbitrarily accurate approximation to a function and its derivatives.*

Keywords—Multilayer feedforward networks, Activation function, Universal approximation capabilities, Input environment measure, $L^p(\mu)$ approximation, Uniform approximation, Sobolev spaces, Smooth approximation.

1. INTRODUCTION

The approximation capabilities of neural network architectures have recently been investigated by many authors, including Carroll and Dickinson (1989), Cybenko (1989), Funahashi (1989), Gallant and White (1988), Hecht-Nielsen (1989), Hornik, Stinchcombe, and White (1989, 1990), Irie and Miyake (1988), Lapedes and Farber (1988), Stinchcombe and White (1989, 1990). (This list is by no means complete.)

If we think of the network architecture as a rule for computing values at l output units given values at k input units, hence implementing a class of mappings from R^k to R^l, we can ask how well arbitrary mappings from R^k to R^l can be approximated by the network, in particular, if as many hidden units as required for internal representation and computation may be employed.

How to measure the accuracy of approximation depends on how we measure closeness between functions, which in turn varies significantly with the specific problem to be dealt with. In many applications, it is necessary to have the network perform *simultaneously* well on all input samples taken from some compact input set X in R^k. In this case, closeness is measured by the uniform distance between functions on X, that is,

$$\rho_{\infty,X}(f, g) = \sup_{x \in X} |f(x) - g(x)|.$$

In other applications, we think of the inputs as random variables and are interested in the *average performance* where the average is taken with respect to the input environment measure μ, where $\mu(R^k) < \infty$. In this case, closeness is measured by the $L^p(\mu)$ distances

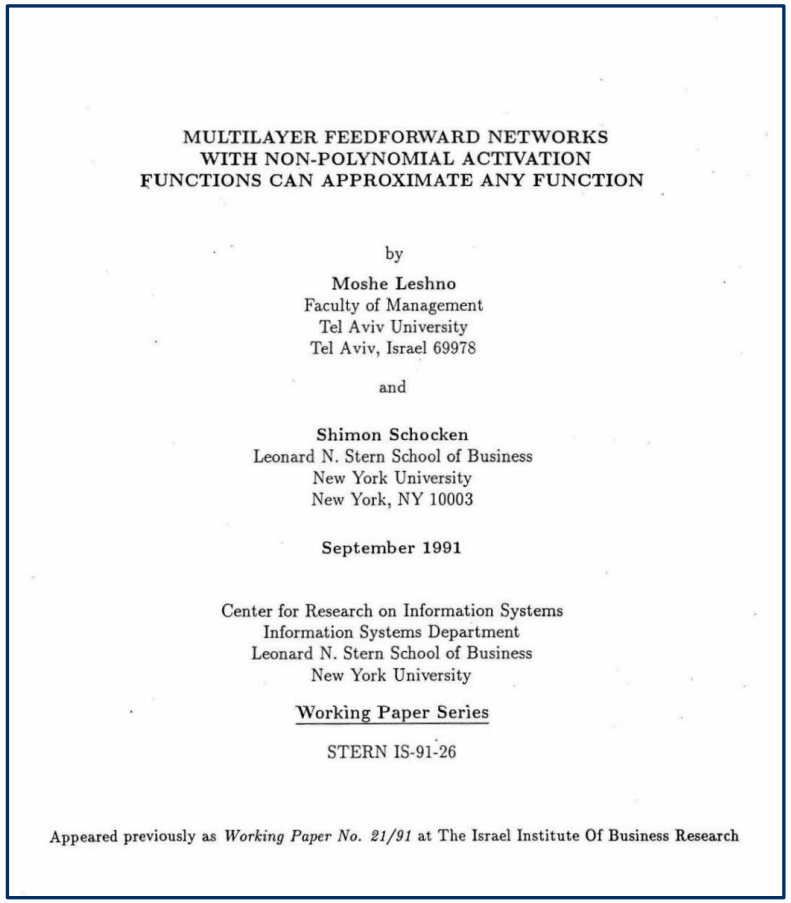
$$\rho_{p,\mu}(f, g) = \left[\int_{R^k} |f(x) - g(x)|^p \, d\mu(x)\right]^{1/p},$$

$1 \le p < \infty$, the most popular choice being $p = 2$, corresponding to mean square error.

Of course, there are many more ways of measuring closeness of functions. In particular, in many applications, it is also necessary that the *derivatives* of the approximating function implemented by the network closely resemble those of the function to be approximated, up to some order. This issue was first taken up in Hornik et al. (1990), who discuss the sources of need of smooth functional approximation in more detail. Typical examples arise in robotics (learning of smooth movements) and signal processing (analysis of chaotic time series); for a recent application to problems of nonparametric inference in statistics and econometrics, see Gallant and White (1989).

All papers establishing certain approximation ca-

Requests for reprints should be sent to Kurt Hornik, Institut für Statistik und Wahrscheinlichkeitstheorie, Technische Universität Wien, Wiedner Hauptstraße 8-10/107, A-1040 Wien, Austria.



MULTILAYER FEEDFORWARD NETWORKS WITH NON-POLYNOMIAL ACTIVATION FUNCTIONS CAN APPROXIMATE ANY FUNCTION

by

Moshe Leshno
Faculty of Management
Tel Aviv University
Tel Aviv, Israel 69978

and

Shimon Schocken
Leonard N. Stern School of Business
New York University
New York, NY 10003

September 1991

Center for Research on Information Systems
Information Systems Department
Leonard N. Stern School of Business
New York University

Working Paper Series

STERN IS-91-26

Appeared previously as *Working Paper No. 21/91* at The Israel Institute Of Business Research

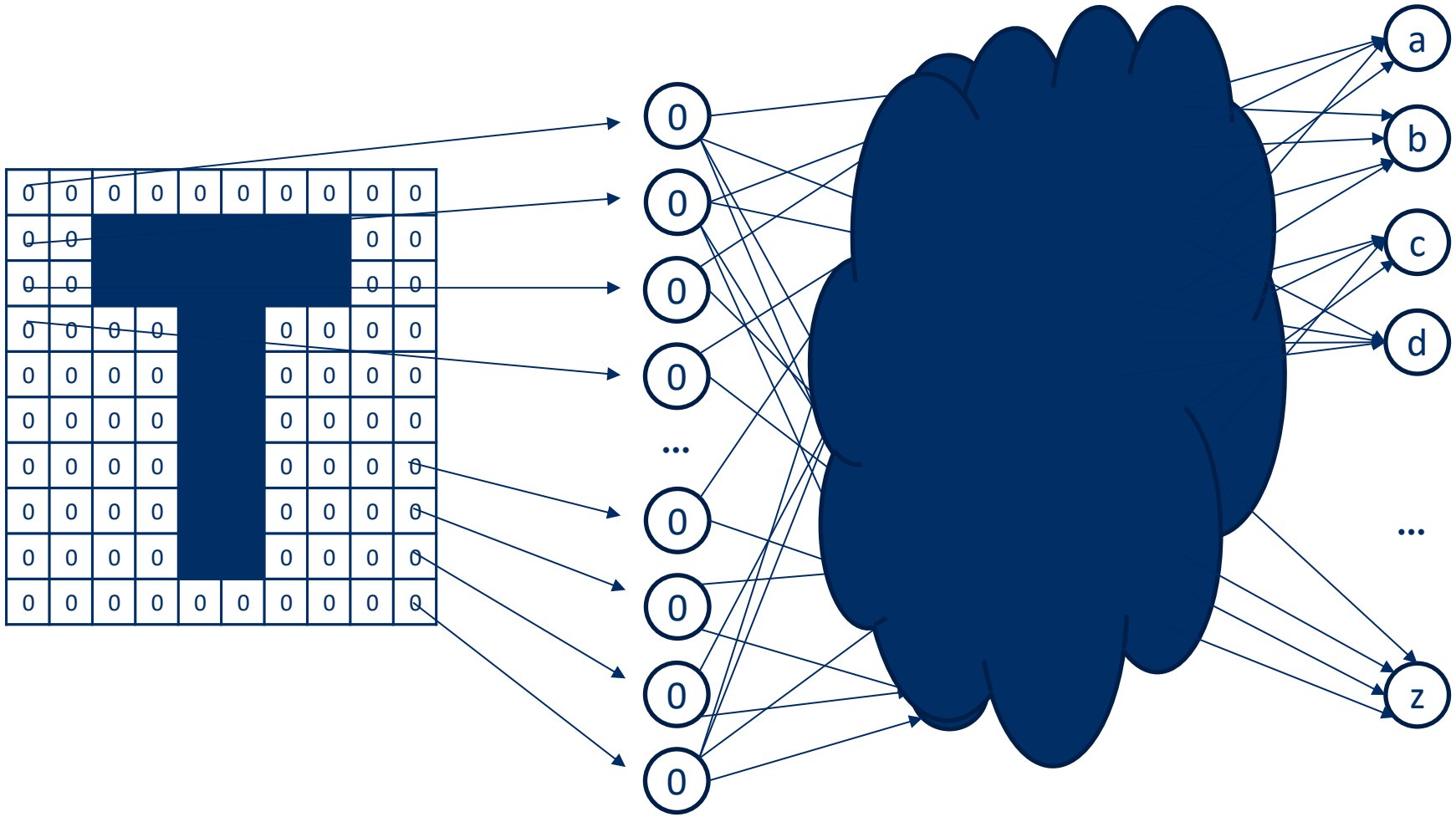
Cybenko (1989) “Approximations by superpositions of sigmoidal functions”
Hornik (1991) “Approximation Capabilities of Multilayer Feedforward Networks”
Leshno and Schocken (1991) ”Multilayer Feedforward Networks with Non-Polynomial Activation Functions Can Approximate Any Function”

Outline

- Perceptron
- Learning as optimization
- Multi-layer perceptrons
- Deep Neural Networks
- Convolutional Neural Networks

Neural Networks for Image Recognition

- **Can use neural network for image classification:**



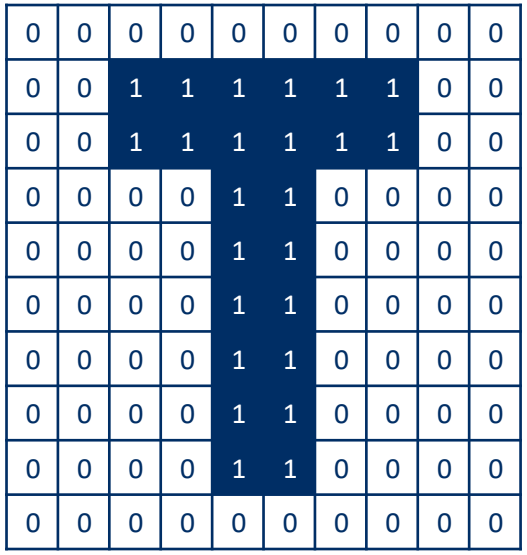
Neural Networks for Image Recognition

- **Big disadvantage of using traditional neural net with fully connected layers:**
- **NN still has to become “spatially aware” during training**
- **There are many, many weights to optimize**
  - May lead to overfitting
  - More weights means more computations

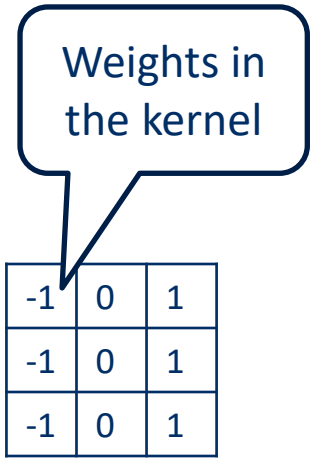
- **Therefore the concept of convolution is introduced**
  - CNNs are state-of-the-art in image recognition

Convolutional Neural Nets

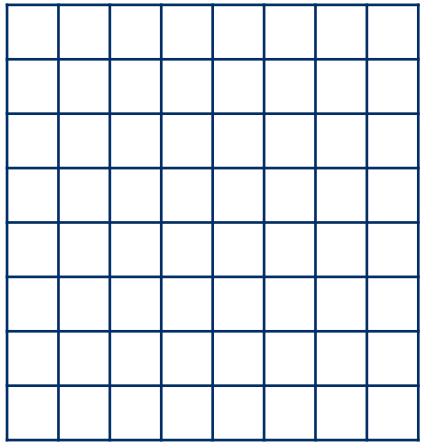
- **Instead of fully connecting two subsequent layers, we use a small *kernel* to connect layers:**



Nodes in layer i



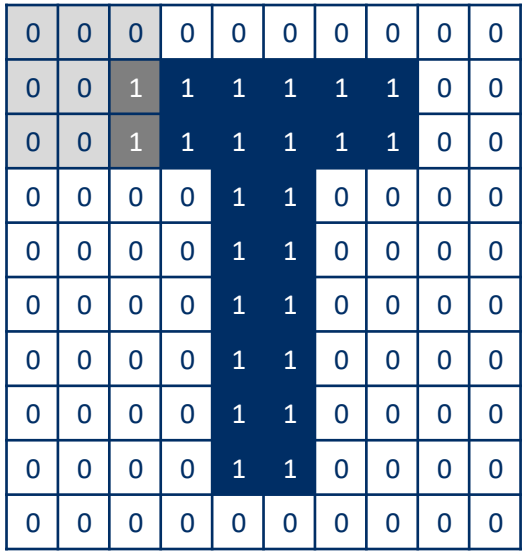
Kernel



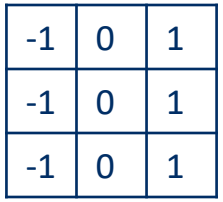
Nodes in layer i+1

Convolutional Neural Nets

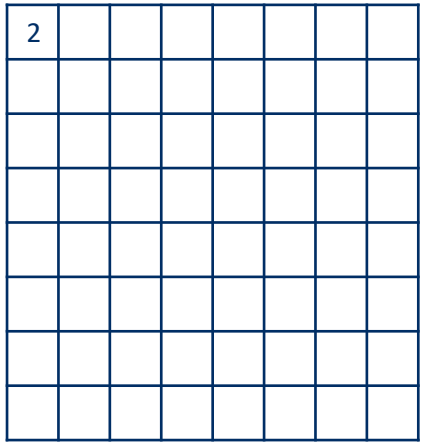
- **Instead of fully connecting two subsequent layers, we use a small *kernel* to connect layers:**



Nodes in layer i



Kernel

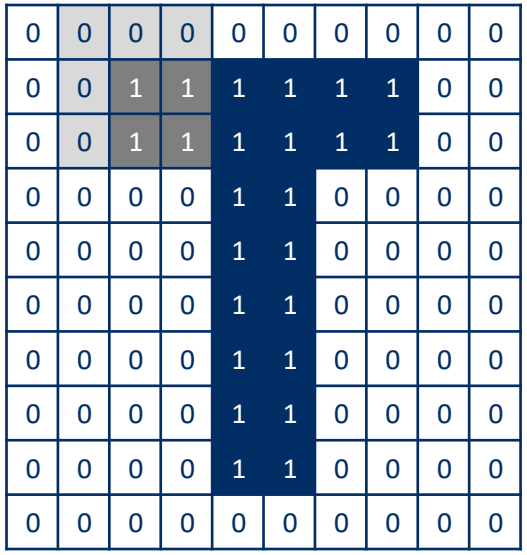


Nodes in layer i+1

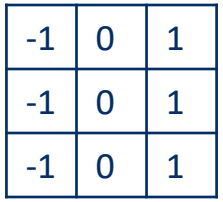
- **Apply Kernel by shifting it over the image**

Convolutional Neural Nets

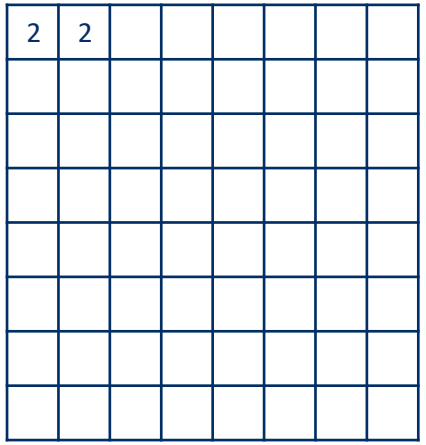
- **Instead of fully connecting two subsequent layers, we use a small *kernel* to connect layers:**



Nodes in layer i



Kernel



Nodes in layer i+1

- **Apply Kernel by shifting it over the image**

Convolutional Neural Nets

- **Instead of fully connecting two subsequent layers, we use a small *kernel* to connect layers:**

0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	1	0	0
0	0	1	1	1	1	1	1	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Nodes in layer i

-1	0	1
-1	0	1
-1	0	1

Kernel

2	2	0					

Nodes in layer i+1

- **Apply Kernel by shifting it over the image**

Convolutional Neural Nets

- **Instead of fully connecting two subsequent layers, we use a small *kernel* to connect layers:**

0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	1	0	0
0	0	1	1	1	1	1	1	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Nodes in layer i

-1	0	1
-1	0	1
-1	0	1

Kernel

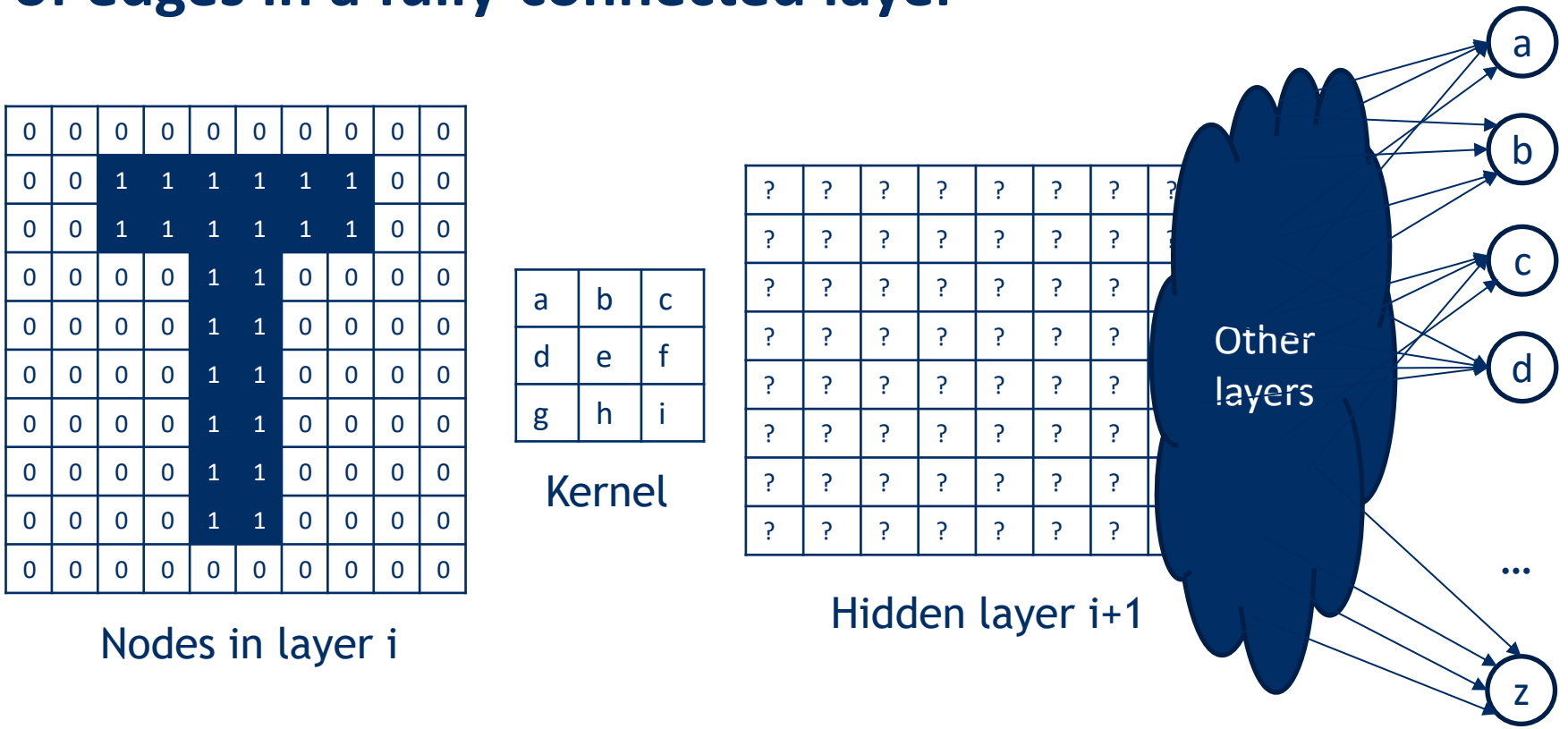
2	2	0	0	0	0	-2	-2
2	2	1	0	0	-1	-2	-2
1	1	2	2	-2	-2	-1	-1
0	0	3	3	-3	-3	0	0
0	0	3	3	-3	-3	0	0
0	0	3	3	-3	-3	0	0
0	0	3	3	-3	-3	0	0
0	0	2	2	-2	-2	0	0

Nodes in layer i+1

- **Apply Kernel by shifting it over the image**

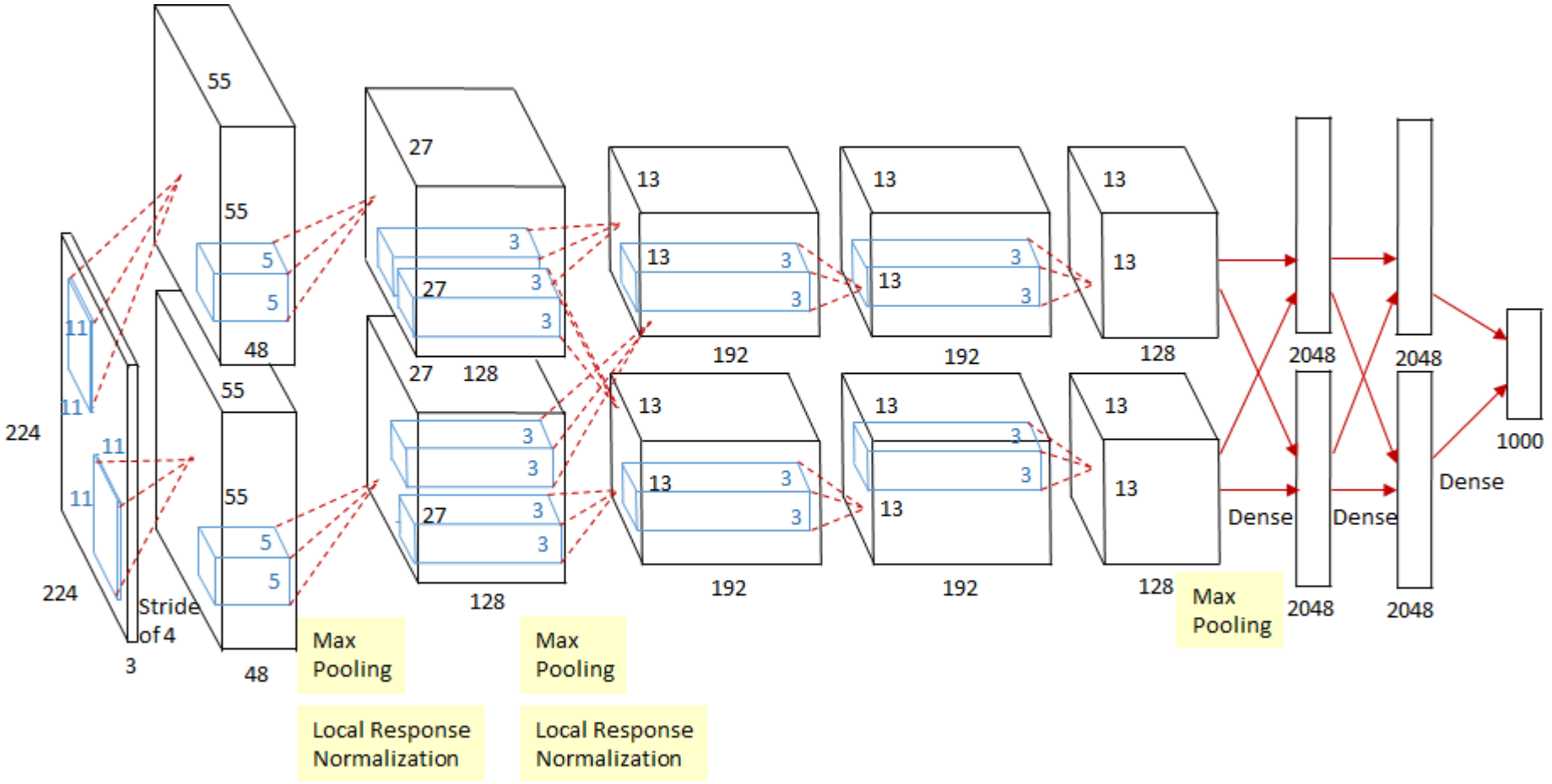
Convolutional Neural Networks

- **The parameters of the Kernel can be learned in a similar way as the weights of edges in a fully-connected layer**



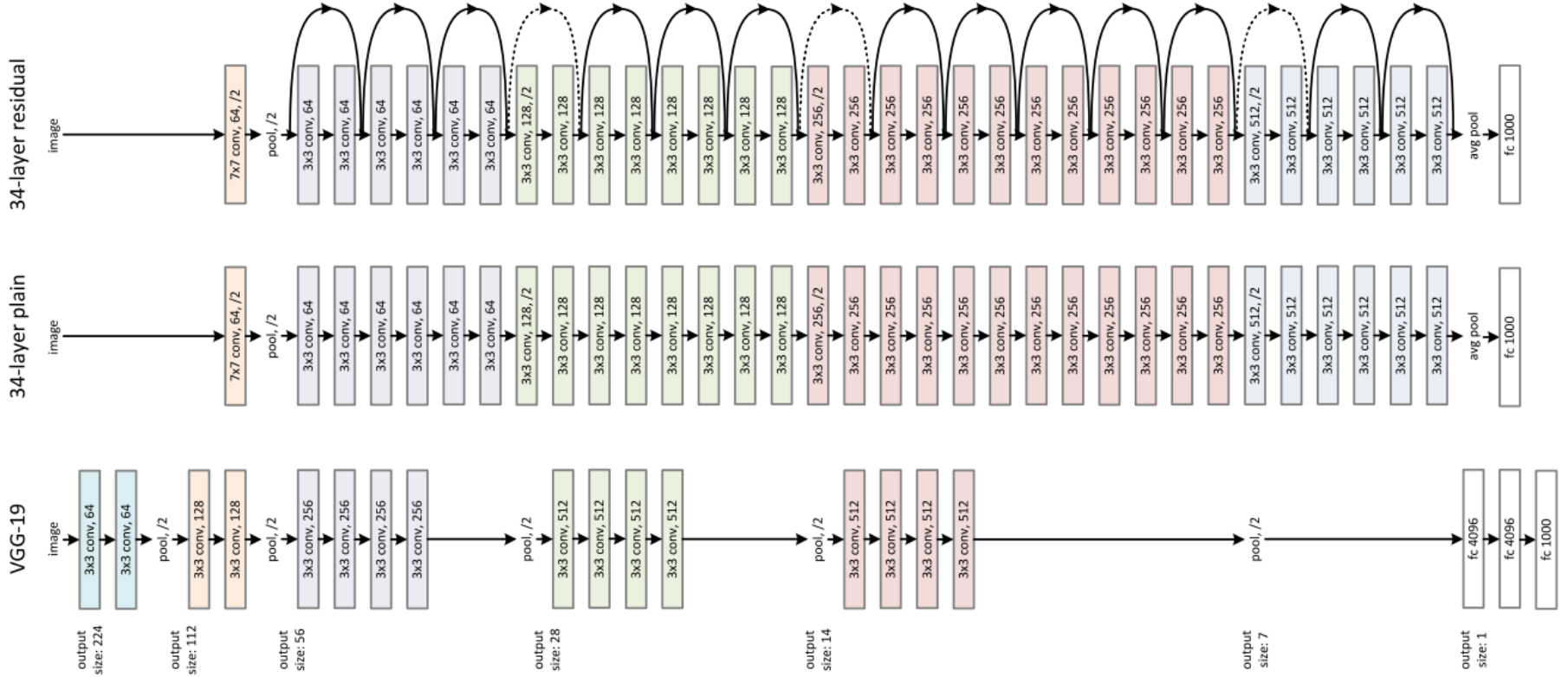
example

Example : AlexNet (2012)



https://medium.com/coinmonks/paper-review-of-alexnet-caffenet-winner-in-ilsvrc-2012-image-classification-b93598314160

Example: ResNet



https://towardsdatascience.com/review-resnet-winner-of-ilsvrc-2015-image-classification-localization-detection-e39402bfa5d8

Neural Networks for Computer Vision

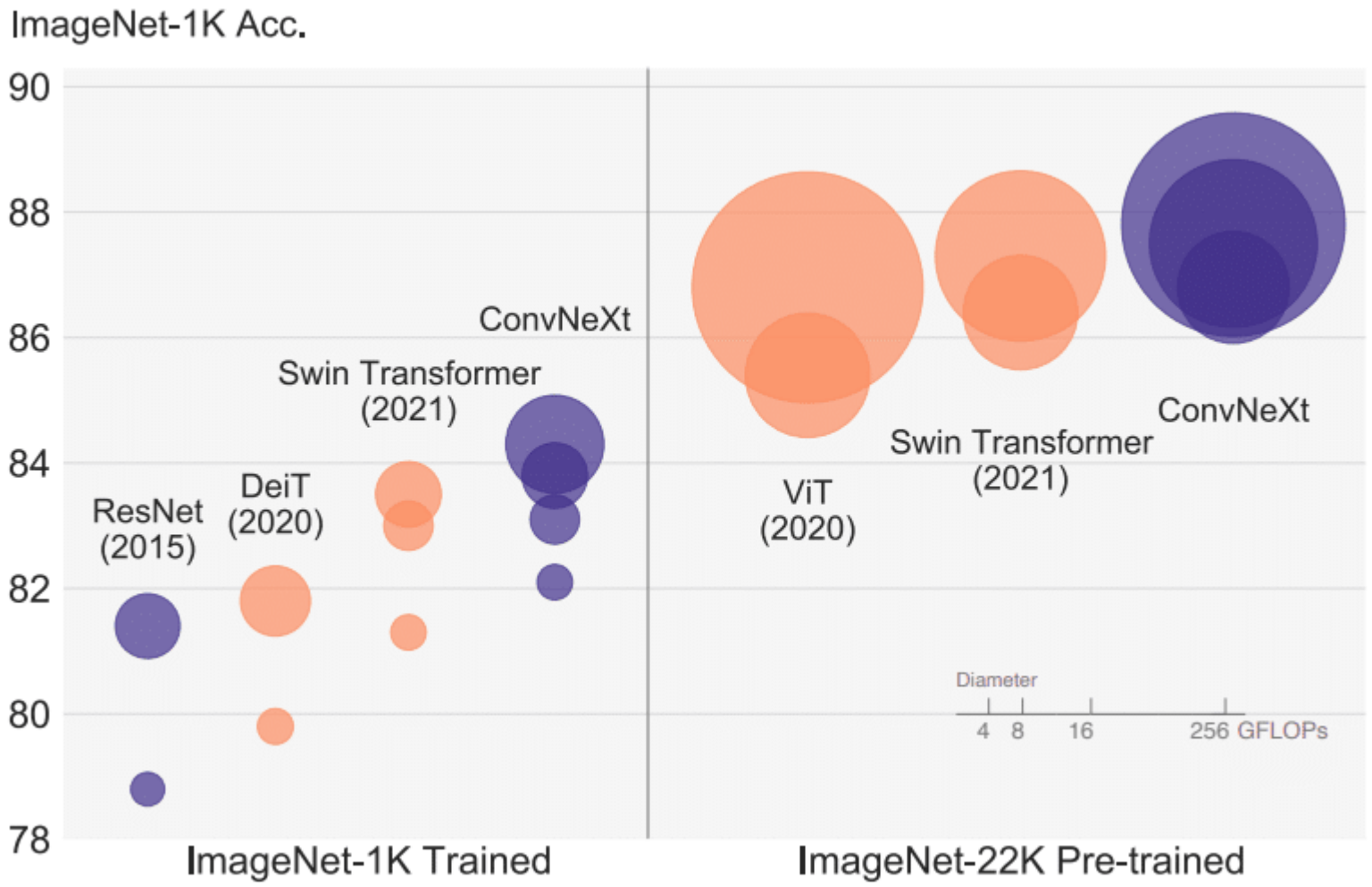


Image source: [A ConvNet for the 2020s]

How to Avoid Overfitting ?

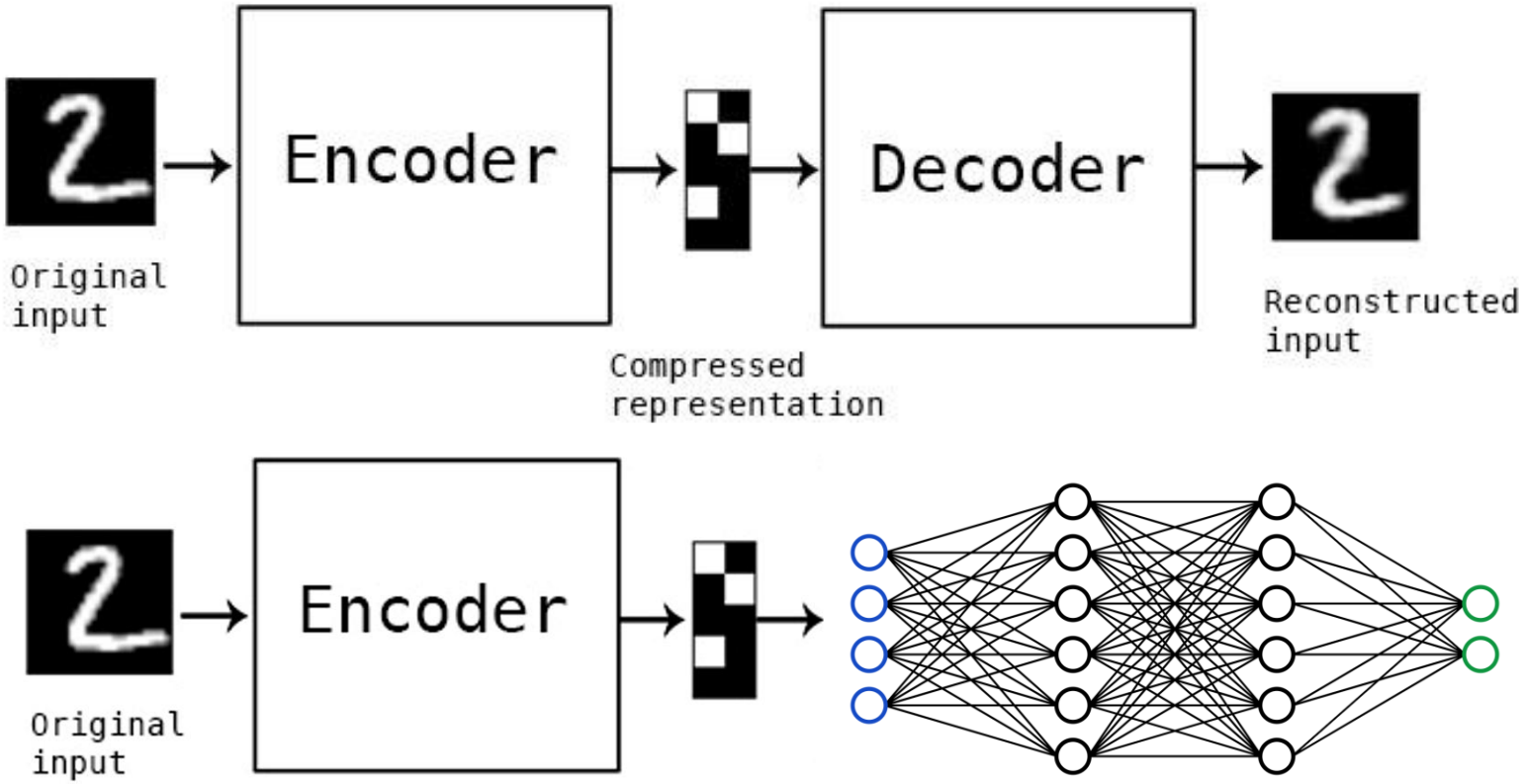
- **Often many millions of parameters to optimize**
- **Number of examples is too low to justify complexity**

- **Several techniques were developed:**
- **Regularization of weights**
  - Add penalty term to objective function
- **Dropout layers**
  - During learning randomly “disable” connections
- **Increase number of training samples**
  - Data augmentation
- **…**

Example

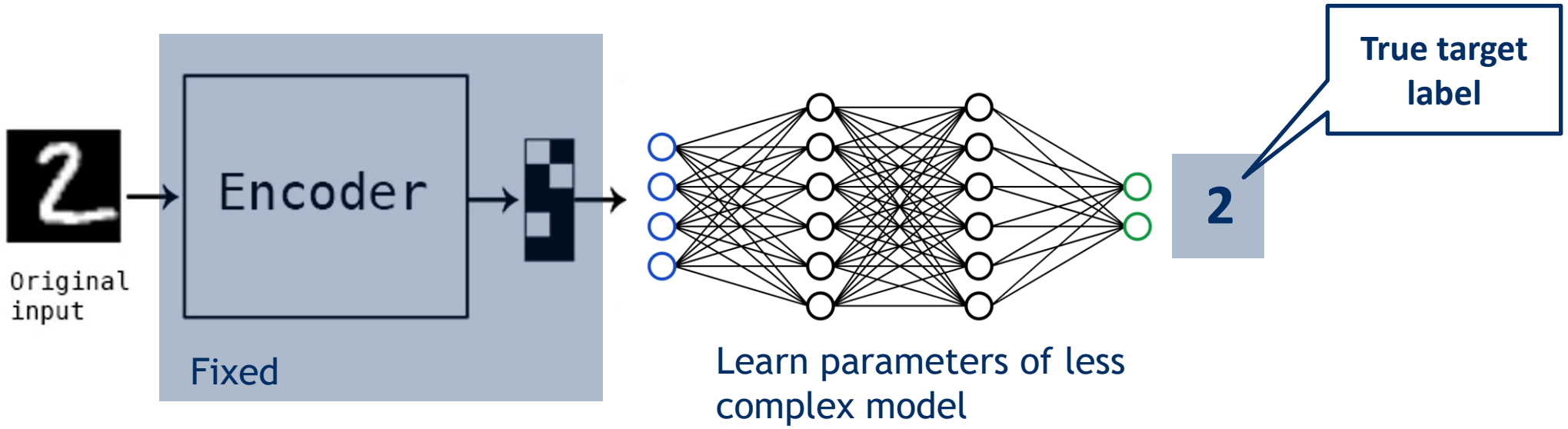
Pre-training

- **Training a network requires a lot of labeled data**
  - Pre-trained on other tasks:



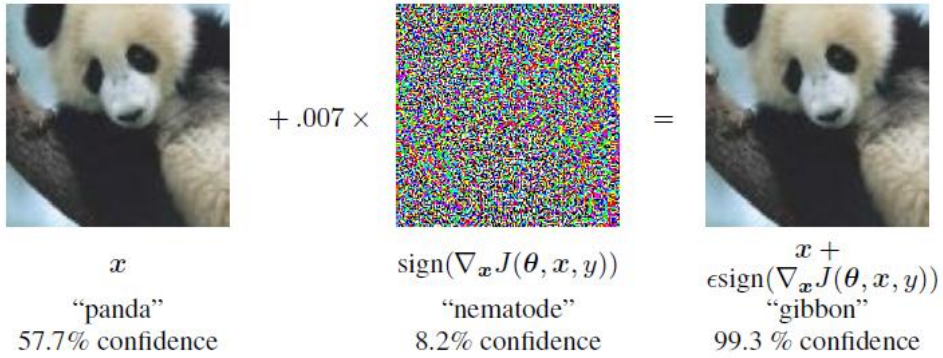
Pre-training

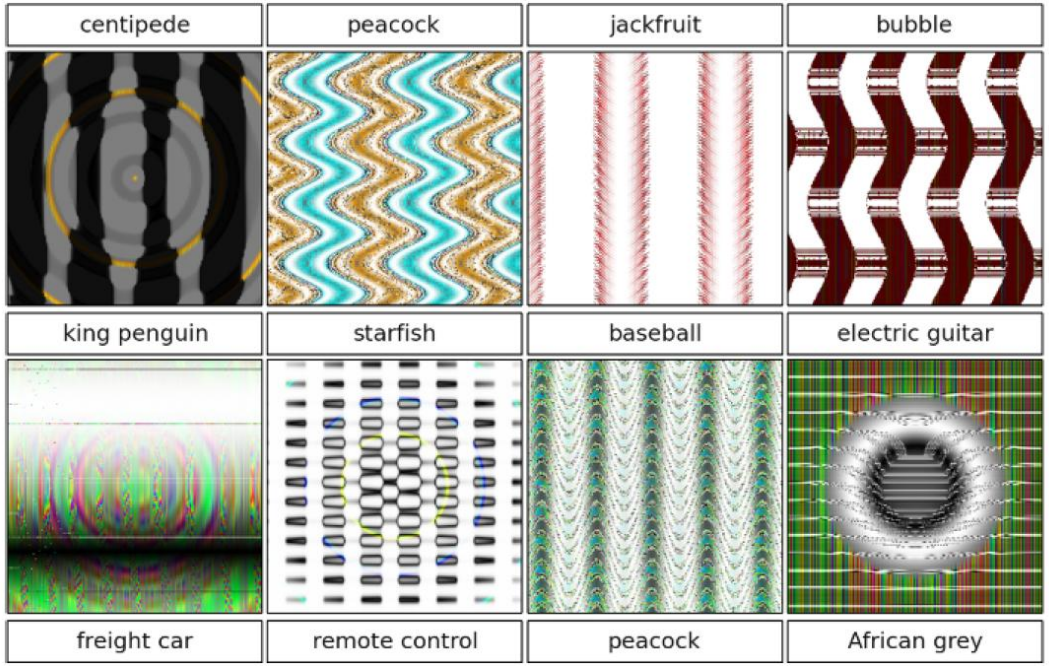
- **Training a network requires a lot of labeled data**
  - Pre-trained on other tasks
  - Fix encoder, learn NN that maps compressed representation to target



But still …

Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. "EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES." *stat* 1050 (2015): 20.

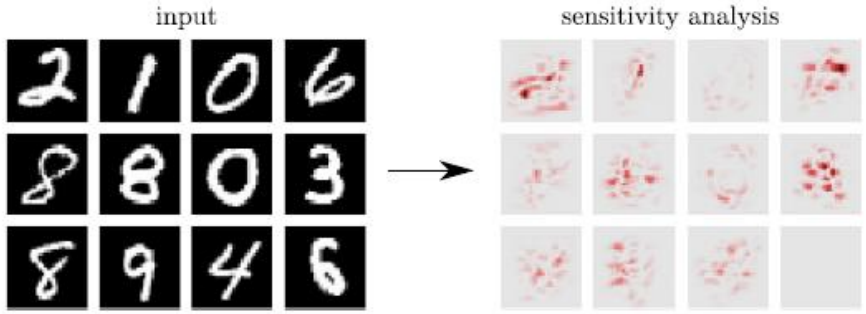


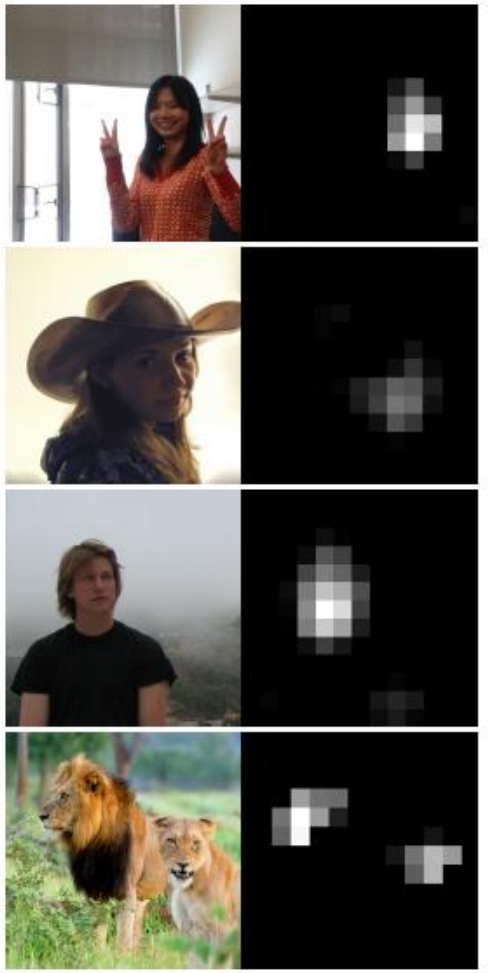


Nguyen A, Yosinski J, Clune J. Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images. In Computer Vision and Pattern Recognition (CVPR '15), IEEE, 2015

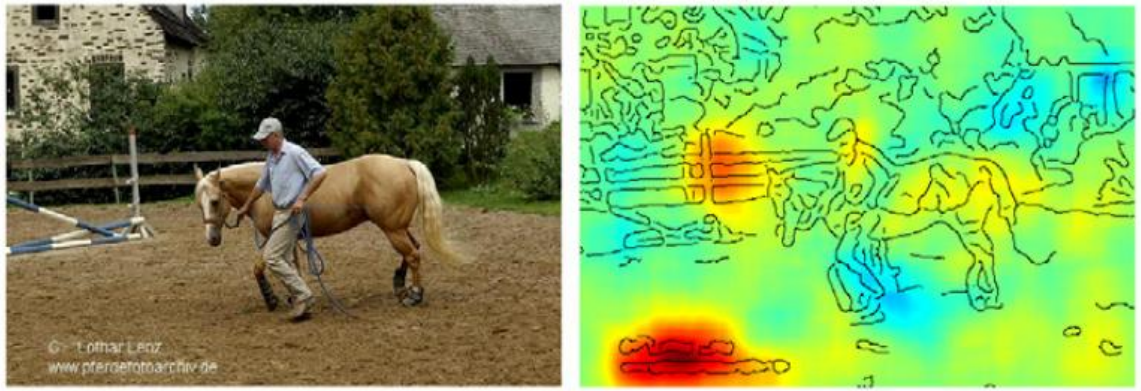
Explaining output of a DNN

Yosinski, J., Clune, J., Nguyen, A., Fuchs, T., & Lipson, H. (2015). Understanding neural networks through deep visualization. *arXiv preprint arXiv:1506.06579*.





Montavon, G., Samek, W., & Müller, K. R. (2018). Methods for interpreting and understanding deep neural networks. *Digital Signal Processing*, *73*, 1-15.



Conclusion

- **Neural networks are a great tool**
  - Especially in domains in which features are hard to construct
- **But they require lots of data to train**
  - Sometimes we can do “data augmentation”; i.e. : create new data points
    - Rotate pictures over a small angle
    - Shift speech in time
- **And lots of computational power**
  - GPUs help
- **However, many trained models are available for free**
  - Can be used if the task is the same, or can be *retrained* for slightly different tasks

